DOCKER - CHALLENGE

1.Create a customized Docker image using a Dockerfile.

This is a basic Docker practical. We will create a custom **Nginx Docker image** using a Dockerfile, add our own HTML page, build the image, run a container, and verify it.

1. Objective

Create a customized Docker image using:

- nginx as the base image
- Custom index.html
- Dockerfile
- Port 80
- Custom Docker image
- Docker container

2. Check Docker Installation

First verify Docker:

docker --version

Example:

Docker version 28.x.x

Check Docker service:

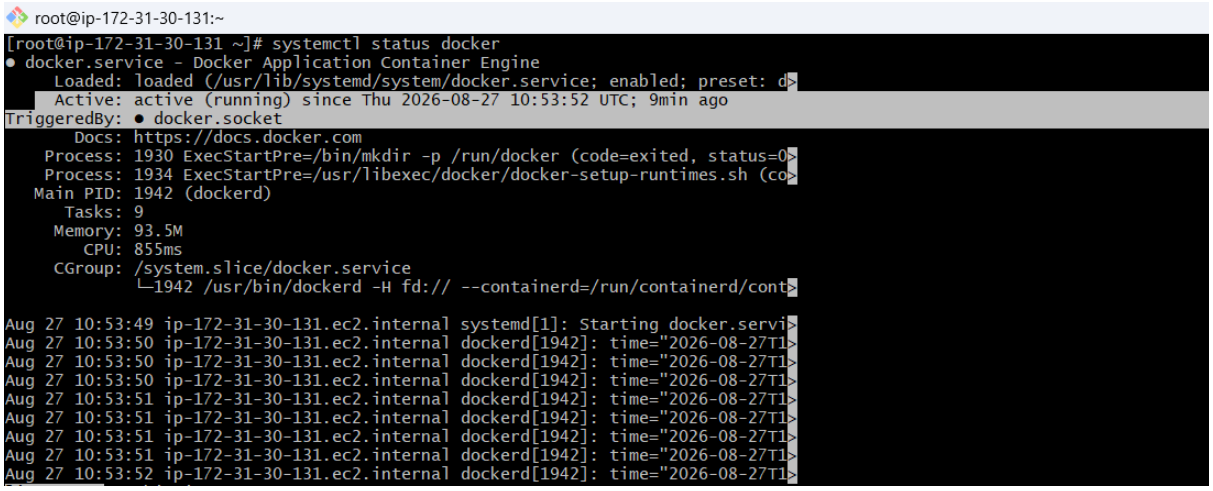
systemctl status docker

If Docker is not running:

systemctl start docker

Enable Docker at boot:

systemctl enable docker



3. Create a Project Directory

Create a directory for the Docker project:

mkdir custom-nginx

Move into it:

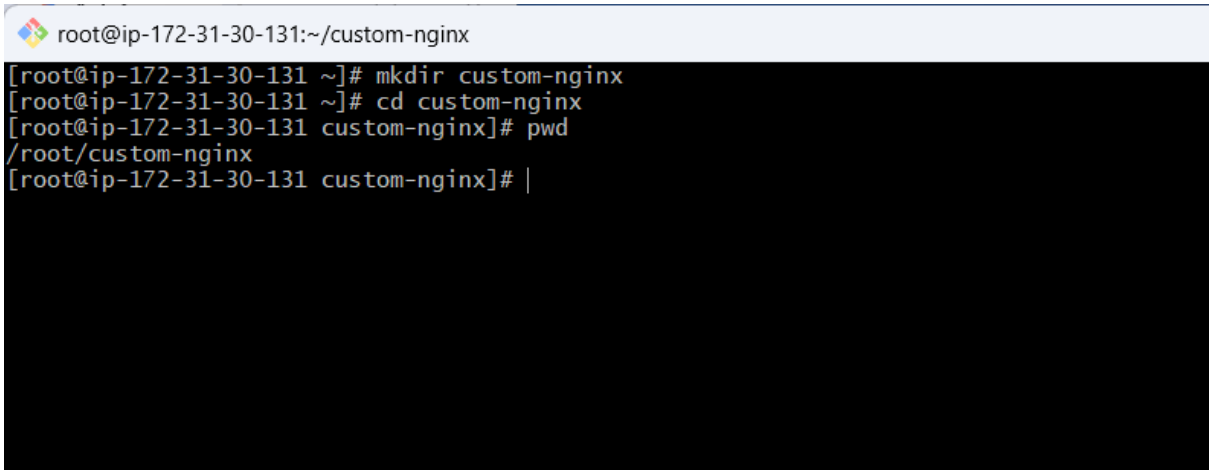
cd custom-nginx

Check:

pwd

Expected:

/root/custom-nginx



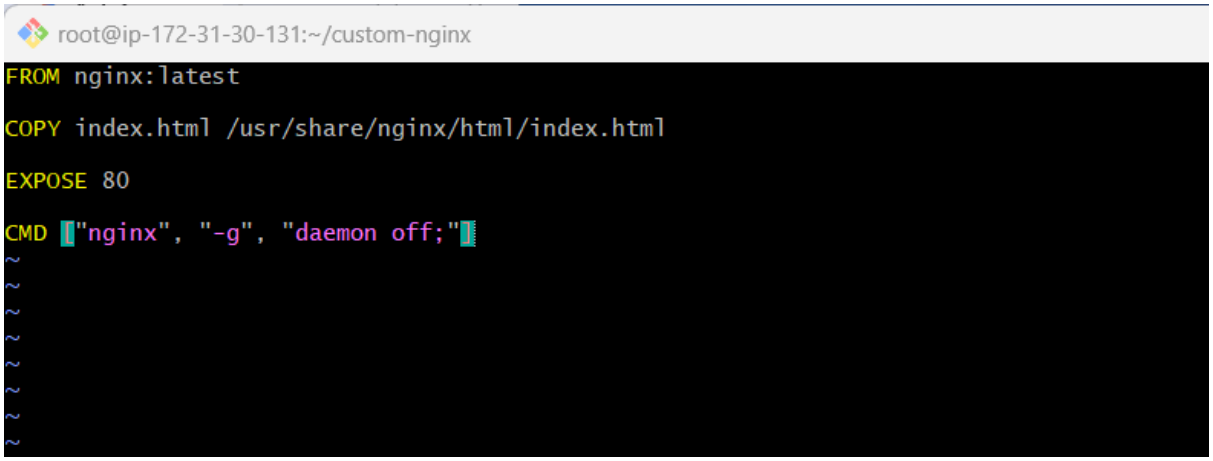
4. Create the Dockerfile

Create a file named exactly:

Dockerfile

Command:

vi Dockerfile



5. Understand the Dockerfile Line by Line

FROM

FROM nginx:latest

This specifies the **base image**.

Docker will use the official Nginx image as the starting point.

nginx:latest

means:

Image = nginx

Tag = latest

COPY

COPY index.html /usr/share/nginx/html/index.html

This copies our local HTML file into the Docker image.

Source:

index.html

Destination:

/usr/share/nginx/html/index.html

Nginx uses this directory to serve web content.

EXPOSE

EXPOSE 80

This documents that the application inside the container listens on port **80**.

Important: EXPOSE does **not** publish the port to the host.

Port publishing happens with:

docker run -p 8080:80 ...

CMD

CMD ["nginx", "-g", "daemon off;"]

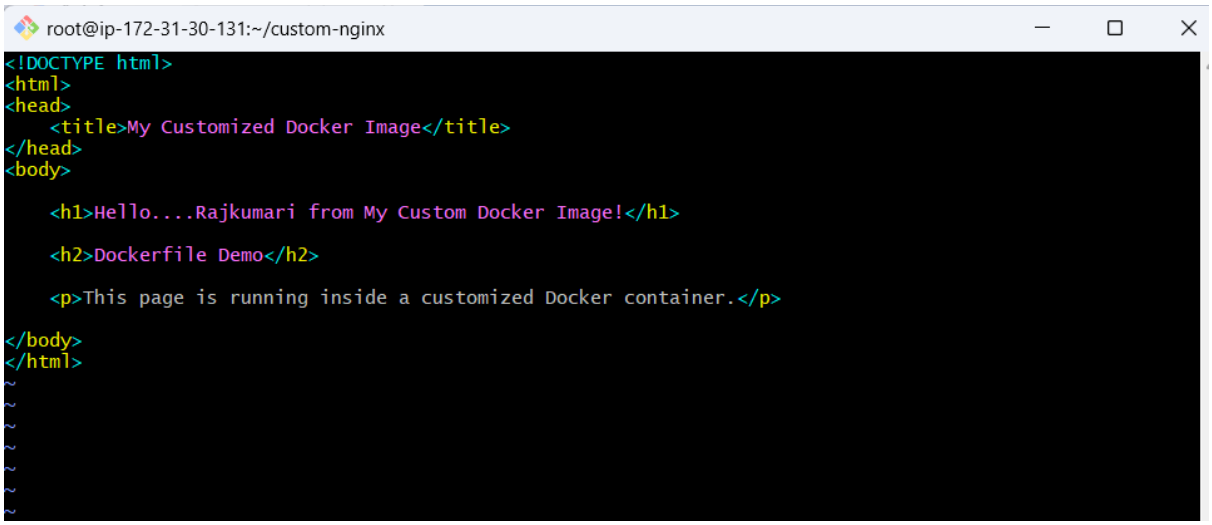
This starts Nginx in the foreground.

The container needs a foreground process to remain running.

6. Create the Custom HTML Page

Create:

vi index.html



7. Verify Project Files

Run:

ls -l

You should see:

Dockerfile

index.html

Your project structure is:

custom-nginx/

│

├── Dockerfile

└── index.html

8. Build the Customized Docker Image

Now build the image:

docker build -t my-custom-nginx:v1 .

Explanation

docker build

Builds a Docker image.

-t

Assigns a name and tag.

my-custom-nginx:v1

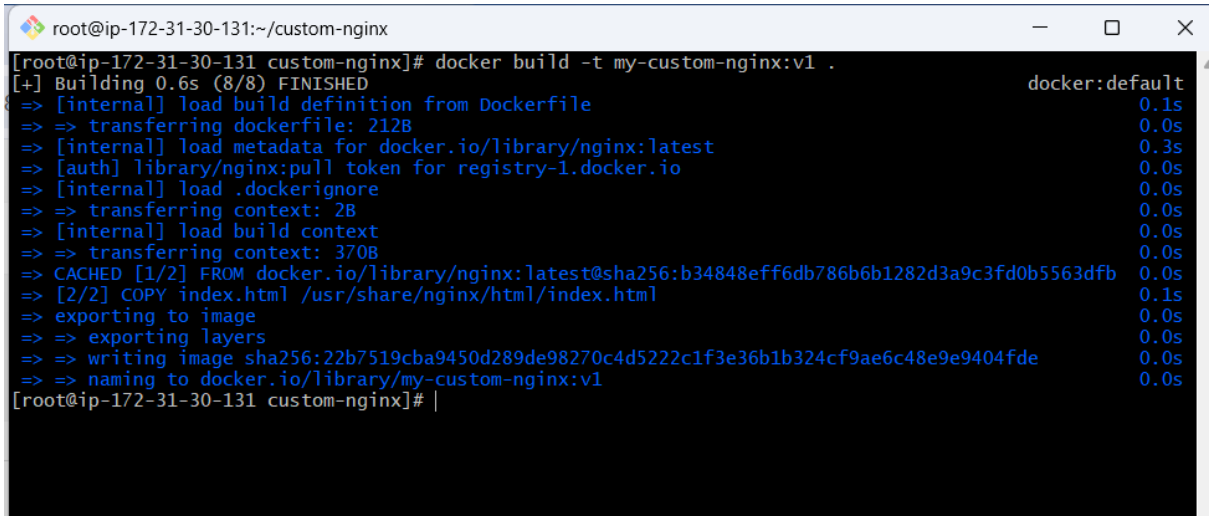
Image name and tag.

.

The current directory is used as the **build context**.

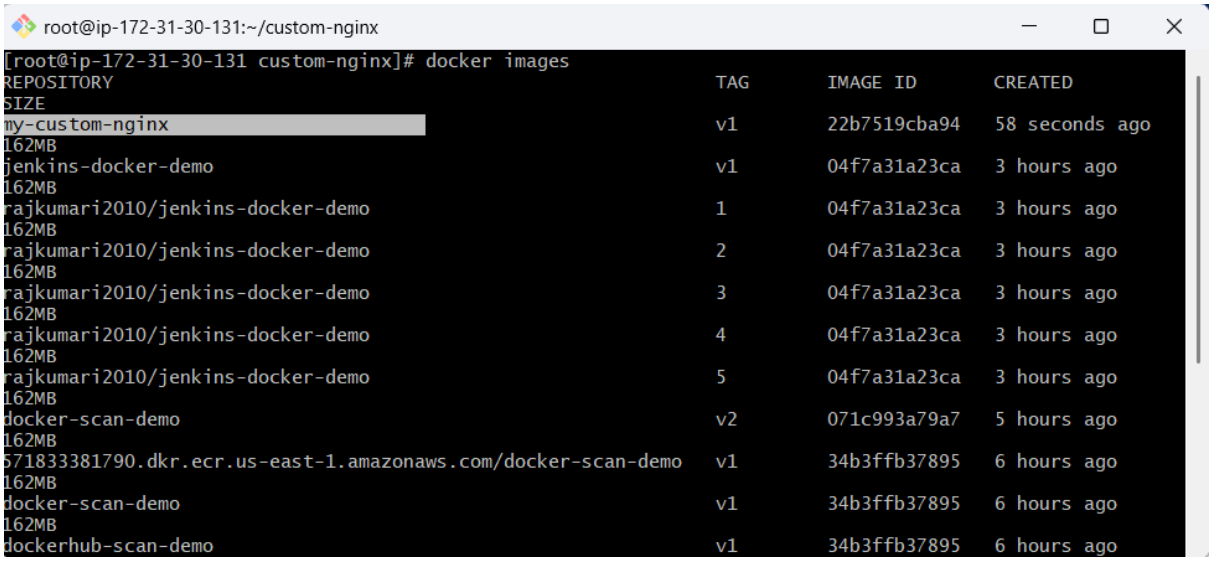
9. Observe the Build

You should see something similar to:



10. Verify the Docker Image

Run: docker images



11. Inspect the Image

You can inspect the image:

docker inspect my-custom-nginx:v1

You can also check its history:

docker history my-custom-nginx:v1

This is useful for understanding the image layers.

You should see a layer corresponding to:

12. Run a Container From Your Customized Image

Run:

```
docker run -d \
  --name custom-nginx-container \
  -p 8080:80 \
  my-custom-nginx:v1
```

Understand the port mapping

-p 8080:80

means:

Host port 8080

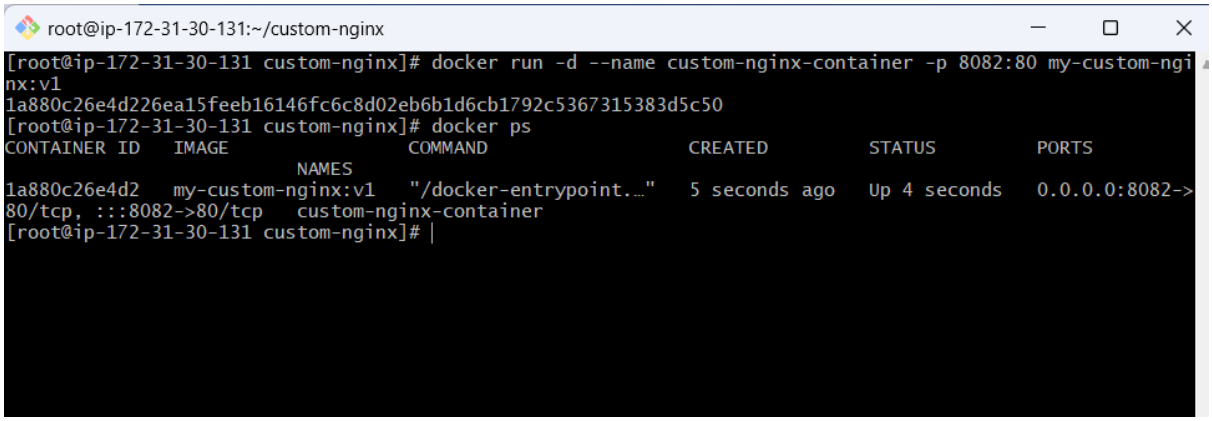
↓

Container port 80

So:

http://<server-ip>:8080

will reach Nginx inside the container



14. Test From Your Browser

If this is an EC2 instance, use:

http://<EC2-Public-IP>:8080

Hello....Rajkumari from My Custom Docker Image!

Dockerfile Demo

This page is running inside a customized Docker container.

6. Verify the Customized File Inside the Container

Find the container:

docker ps

Then execute a command inside it:

docker exec -it custom-nginx-container /bin/bash

If Bash isn't available, try:

docker exec -it custom-nginx-container /bin/sh

Then:

cat /usr/share/nginx/html/index.html

You should see your customized HTML.

Exit:

2. Push the same image to Docker Hub.

Push the Same Docker Image to Docker Hub

Since you already created the customized image:

my-custom-nginx:v1

we can push **the same image** to Docker Hub without rebuilding it.

The process is:

Existing Local Image

|

↓

Docker Tag

|

↓

Docker Hub Login

|

↓

Docker Push

|

↓

Docker Hub

1. Verify the Existing Image

First check your local images:

docker images

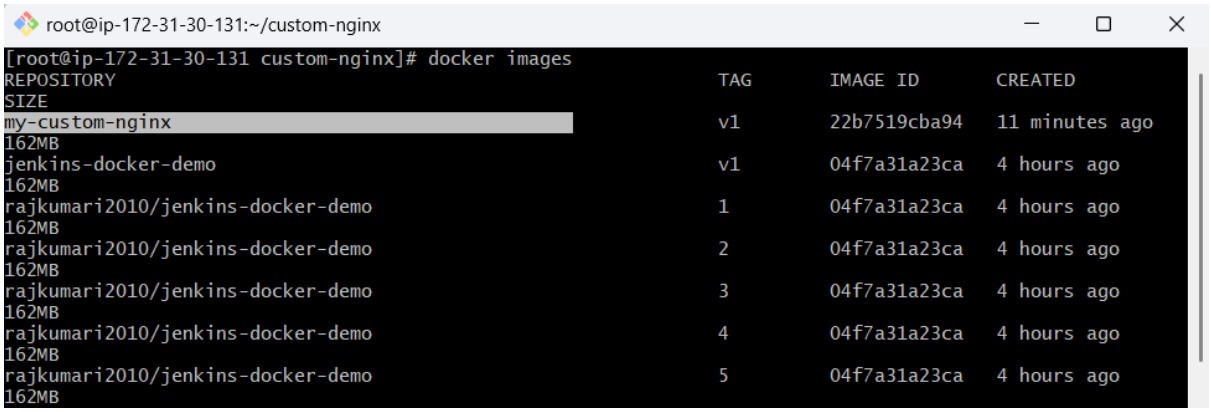
You should see something similar to:

REPOSITORY TAG IMAGE ID CREATED SIZE

my-custom-nginx v1 abc123456789 5 minutes ago ...

The important part is:

my-custom-nginx:v1



2. Create a Docker Hub Repository

Log in to Docker Hub:

Docker Hub

Go to:

Docker Hub

↓

Repositories

↓

Create repository

Create a repository named:

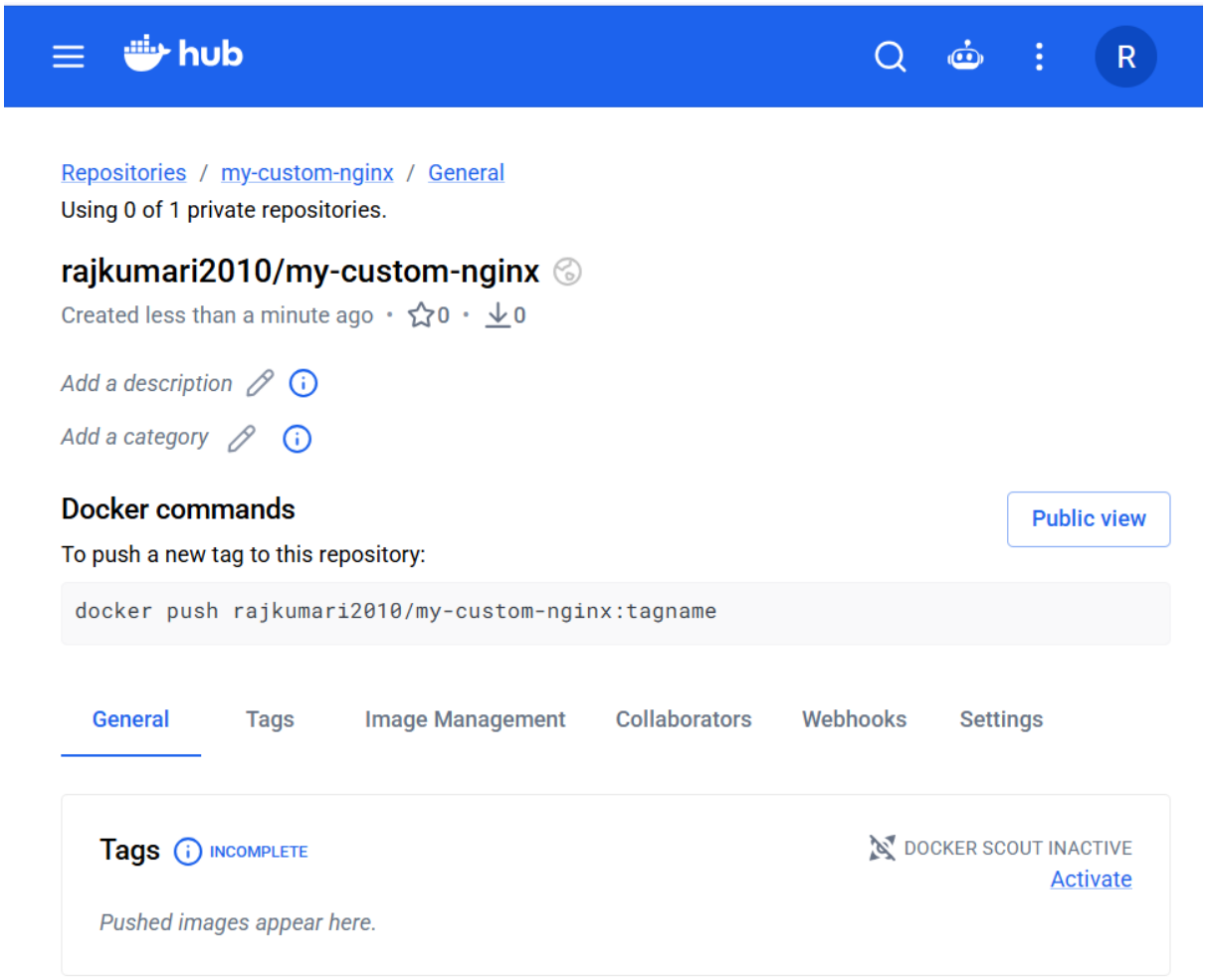
my-custom-nginx

For example, if your Docker Hub username is:

rajkumari

your repository will be:

rajkumari/my-custom-nginx



3. Login to Docker Hub

On your Docker server:

docker login

Enter:

Username: <your-dockerhub-username>

Password: <your-Docker-Hub-PAT>

For example:

Username: rajkumari

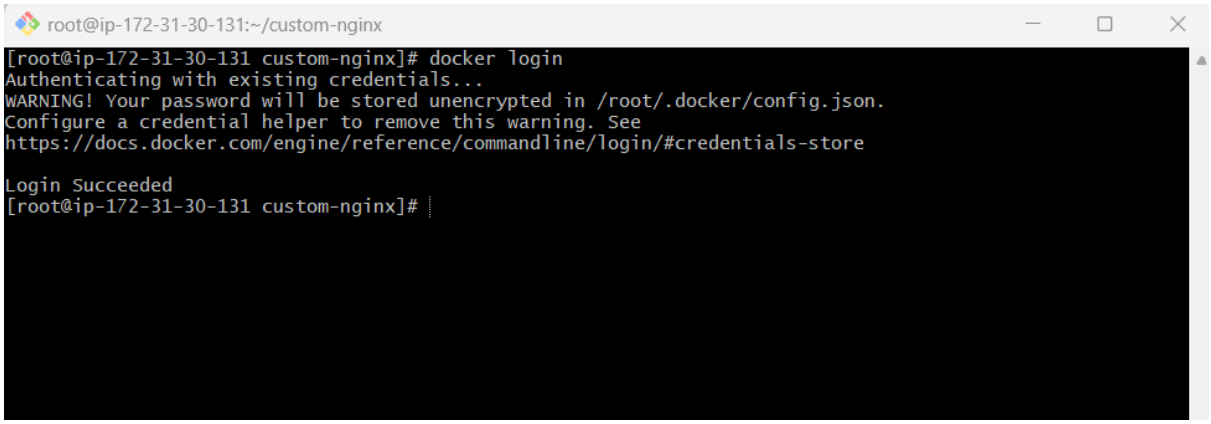
Password: ********

Expected:

Login Succeeded

Important

Use a **Docker Hub Personal Access Token (PAT)** instead of putting your Docker Hub account password into scripts.



4. Tag the Existing Image

This is the most important step.

Your current image is:

my-custom-nginx:v1

Docker Hub requires the image name to include your Docker Hub username.

Run:

docker tag my-custom-nginx:v1 \

<dockerhub-username>/my-custom-nginx:v1

For example:

docker tag my-custom-nginx:v1 \

rajkumari/my-custom-nginx:v1

What happened?

You did **not** create another image.

You simply created another **tag/reference** pointing to the same image.

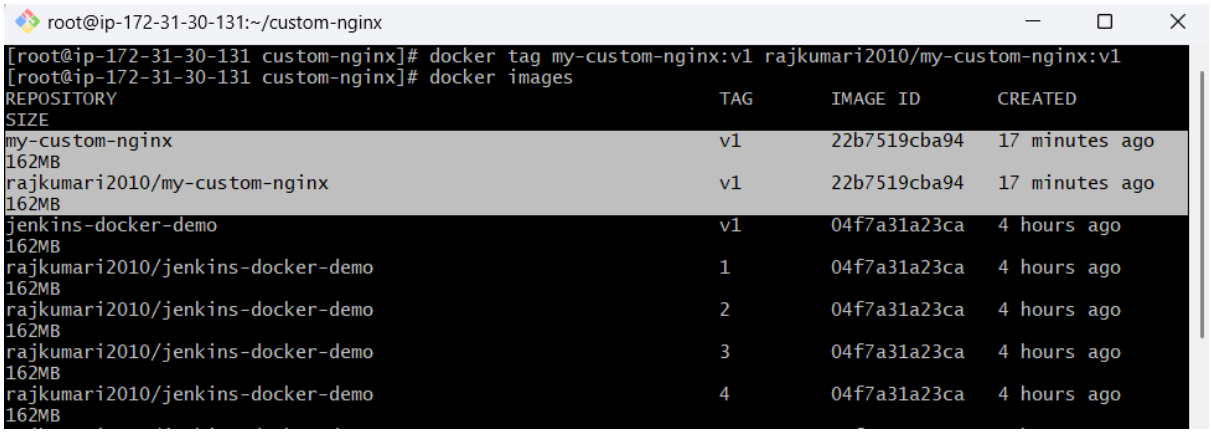
my-custom-nginx:v1

|

| same IMAGE ID

↓

rajkumari/my-custom-nginx:v1

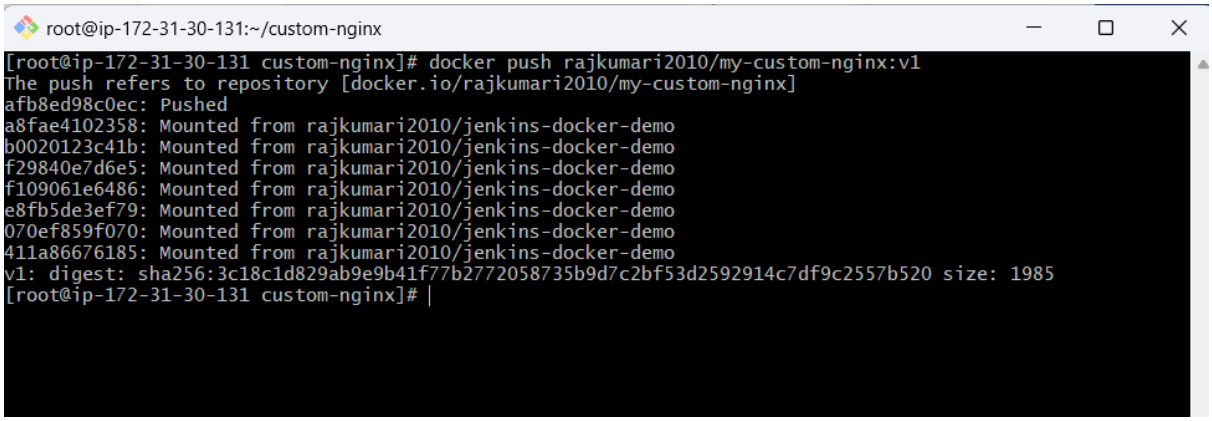


6. Push the Image to Docker Hub

Now run:

docker push rajkumari/my-custom-nginx:v1

Replace rajkumari with your Docker Hub username.



7. Verify in Docker Hub

Go to:

Docker Hub Repositories

Navigate to:

Repositories

↓

my-custom-nginx

↓

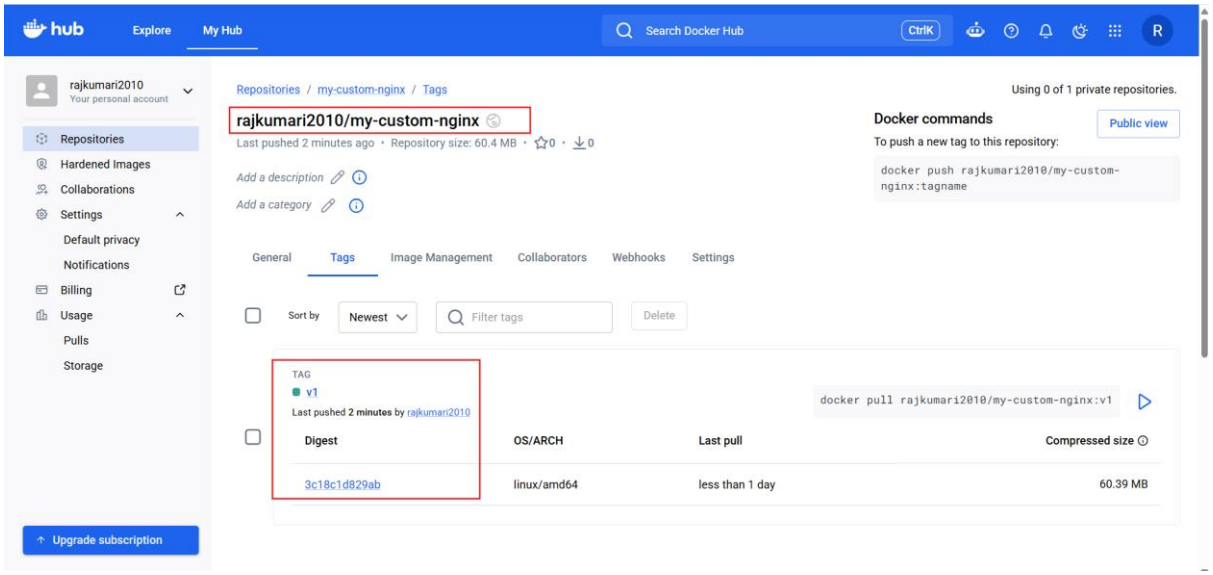
Tags

You should see:

v1

Your image is now:

rajkumari/my-custom-nginx:v1



3. Push the same image to Amazon ECR.

1. Verify the Existing Image

First, check your local Docker images:

docker images

You should see:

REPOSITORY TAG IMAGE ID

my-custom-nginx v1 abc123456789

If you already tagged it for Docker Hub, you may see:

REPOSITORY TAG IMAGE ID

my-custom-nginx v1 abc123456789

rajkumari/my-custom-nginx v1 abc123456789

The important point is that the **IMAGE ID is the same**.

2. Verify AWS CLI

Run:

aws --version

Then check your AWS identity:

aws sts get-caller-identity

3. Set Your AWS Region

For this example, we'll use:

us-east-1

You can verify your current AWS region:

aws configure get region

If required:

aws configure set region us-east-1

4. Create an ECR Repository

Create a repository:

```
aws ecr create-repository \
  --repository-name my-custom-nginx \
  --region us-east-1
```

Expected response will contain something like:

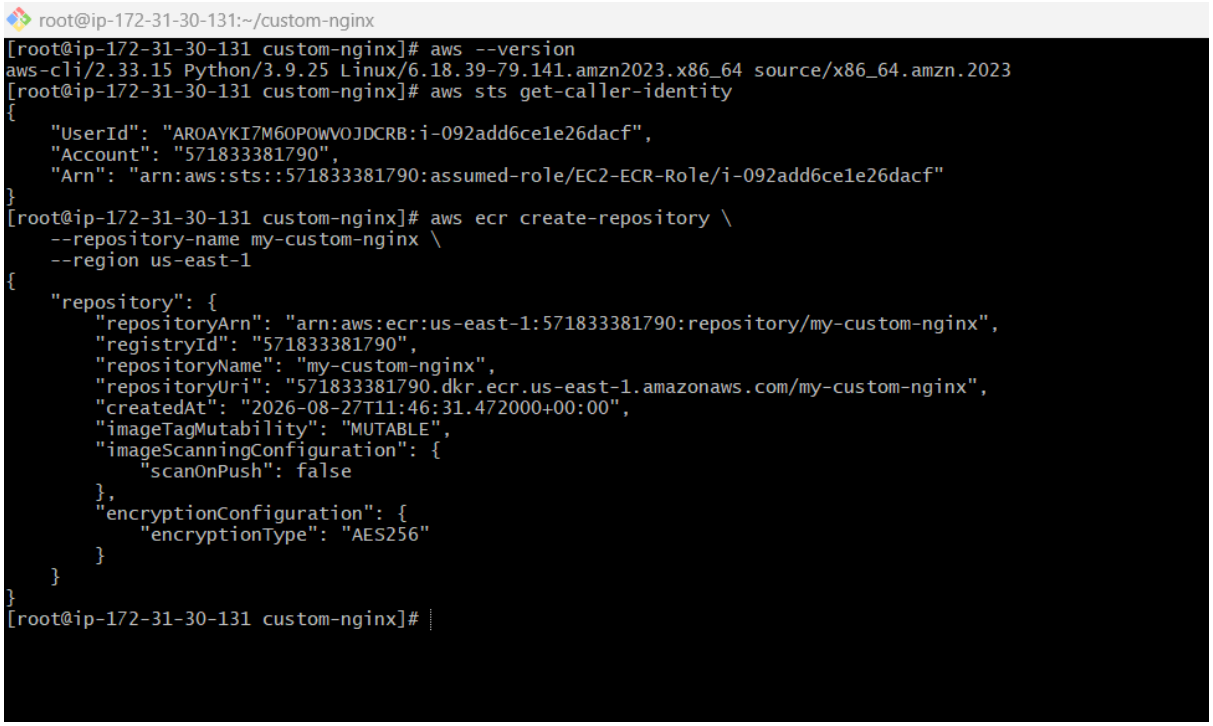
repositoryUri:

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx

Your ECR repository URL is:

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx

Replace 123456789012 with your AWS Account ID.



5. Verify the ECR Repository

Run:

aws ecr describe-repositories \

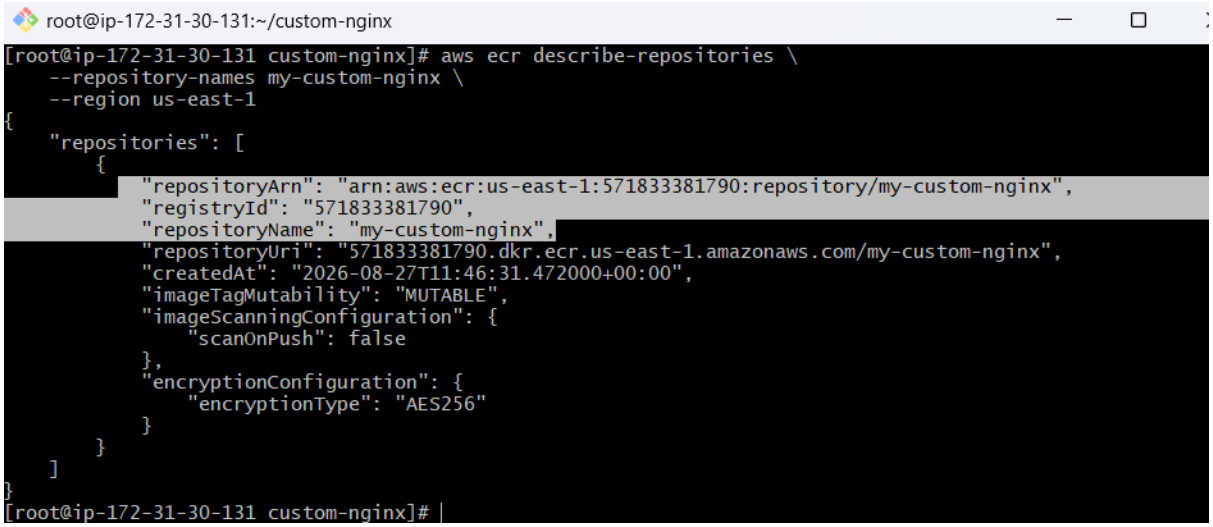
--repository-names my-custom-nginx \

--region us-east-1

You should see:

repositoryName: my-custom-nginx

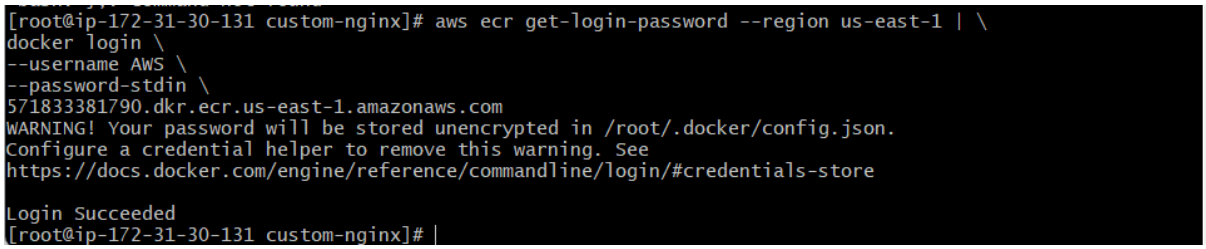
repositoryUri: 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx



6. Login to Amazon ECR

This is an important step.

Run:



7. Tag the Same Image for ECR

Your existing image is:

my-custom-nginx:v1

Tag it with your ECR repository URL:

docker tag my-custom-nginx:v1 \

123456789012.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1

Replace:

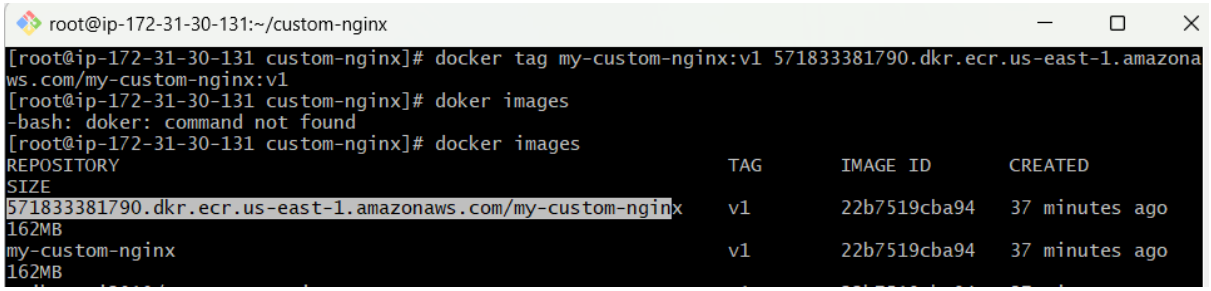
123456789012

with your AWS Account ID.

8. Verify the Image Tags

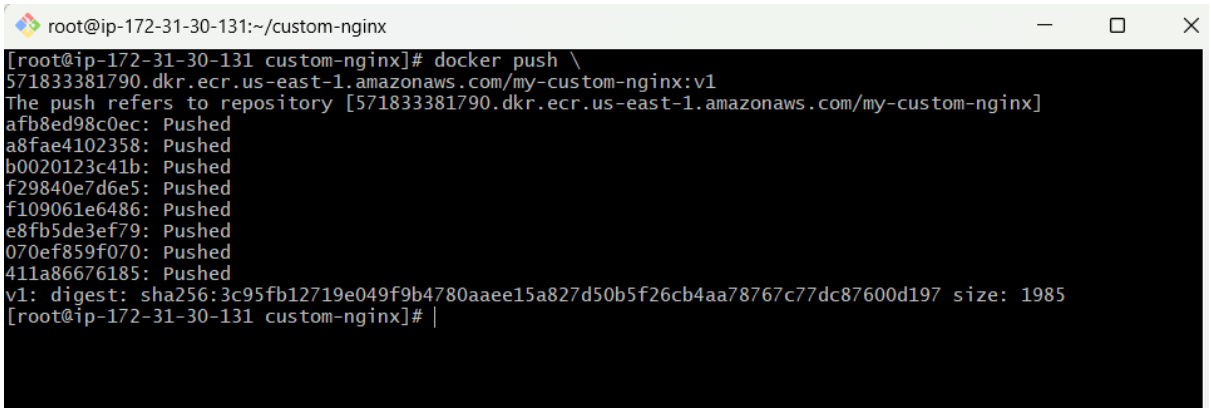
Run:

docker images



9. Push the Image to ECR

Now push: docker push <aws account ID>.dkr.ecr.us-east-1.amazonaws.com/my-custom-nginx:v1



10. Verify the Image in ECR

Run:

aws ecr list-images \

--repository-name my-custom-nginx \

--region us-east-1

11. Get Detailed Image Information

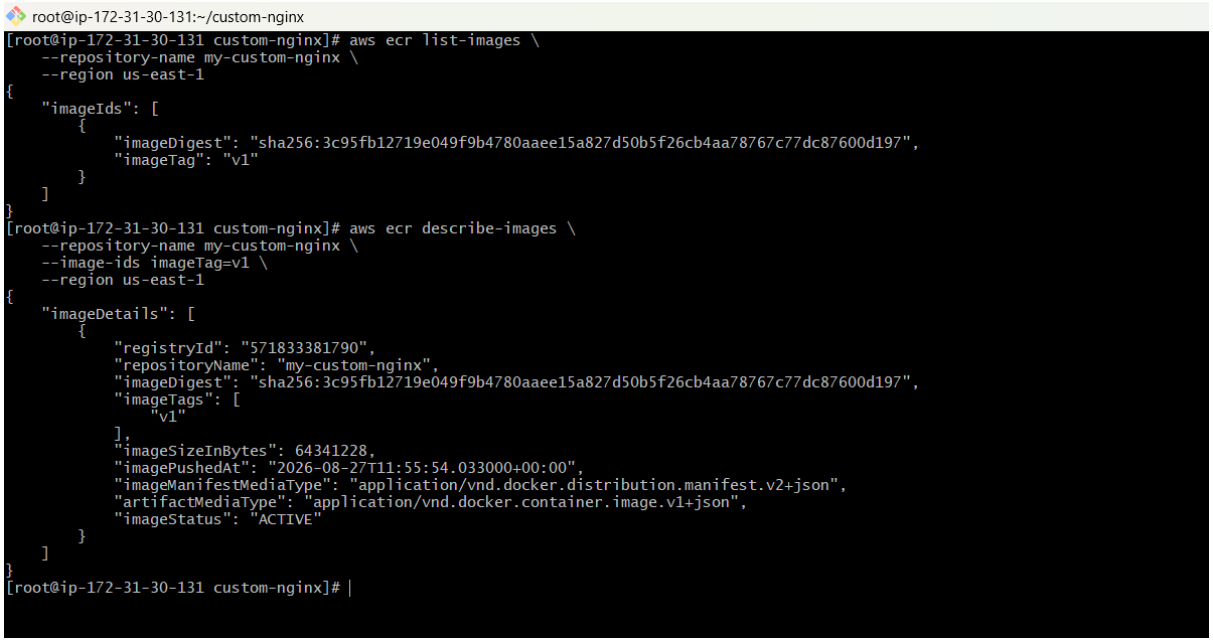
Run:

aws ecr describe-images \

--repository-name my-custom-nginx \

--image-ids imageTag=v1 \

--region us-east-1



12. Verify in AWS Console

Open:

AWS Console

↓

Amazon ECR

↓

Repositories

↓

my-custom-nginx

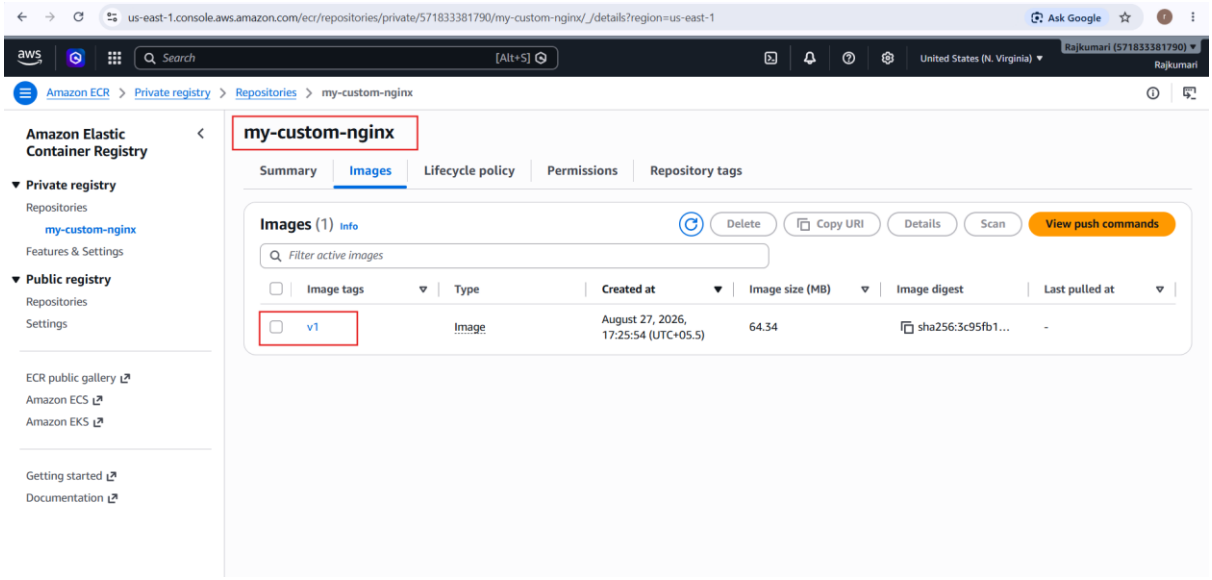
You should see:

Image tag

---

v1

Click the image to view its details.



4. Provision one EC2 instance using Terraform and install Jenkins.

1. Prerequisites

You need:

- AWS account
- AWS IAM permissions to create EC2/security groups
- AWS CLI
- Terraform
- SSH client
- An EC2 key pair

Check Terraform:

terraform --version

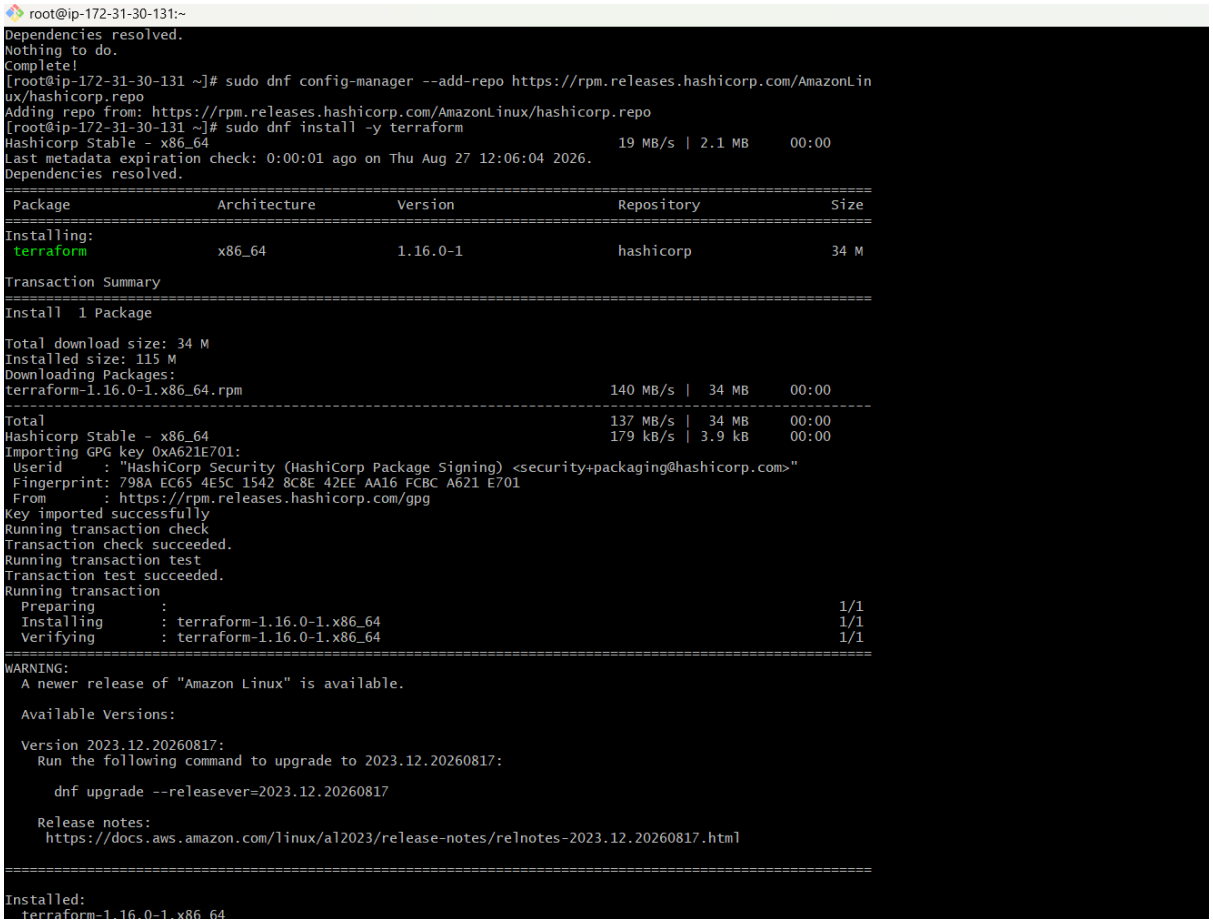
Check AWS CLI:

aws --version

Check AWS credentials:

aws sts get-caller-identity

If you're working from an AWS EC2 server that already has an IAM role, the AWS CLI can use that role.



2. Create a Terraform Project

Create a directory:

mkdir terraform-jenkins

cd terraform-jenkins

Check:

pwd

Project structure will eventually be:

terraform-jenkins/

|

├── provider.tf

├── variables.tf

├── main.tf

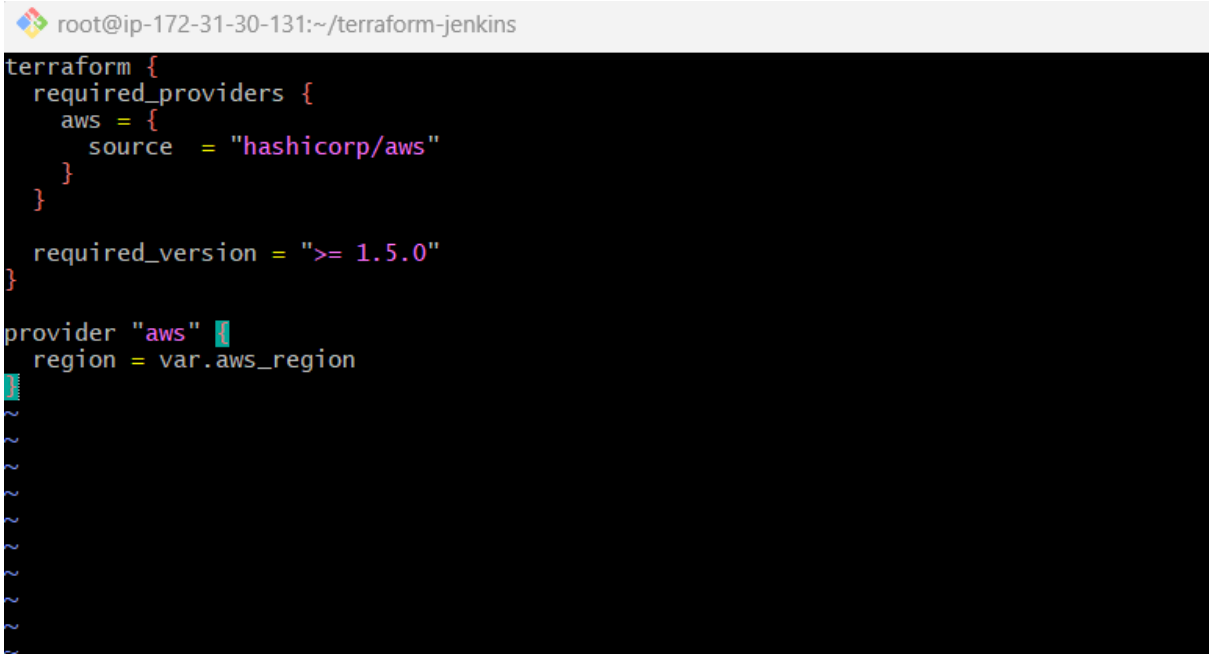
├── outputs.tf

└── userdata.sh

3. Create provider.tf

Create:

vi provider.tf



Explanation

The provider tells Terraform that we are using AWS.

provider "aws"

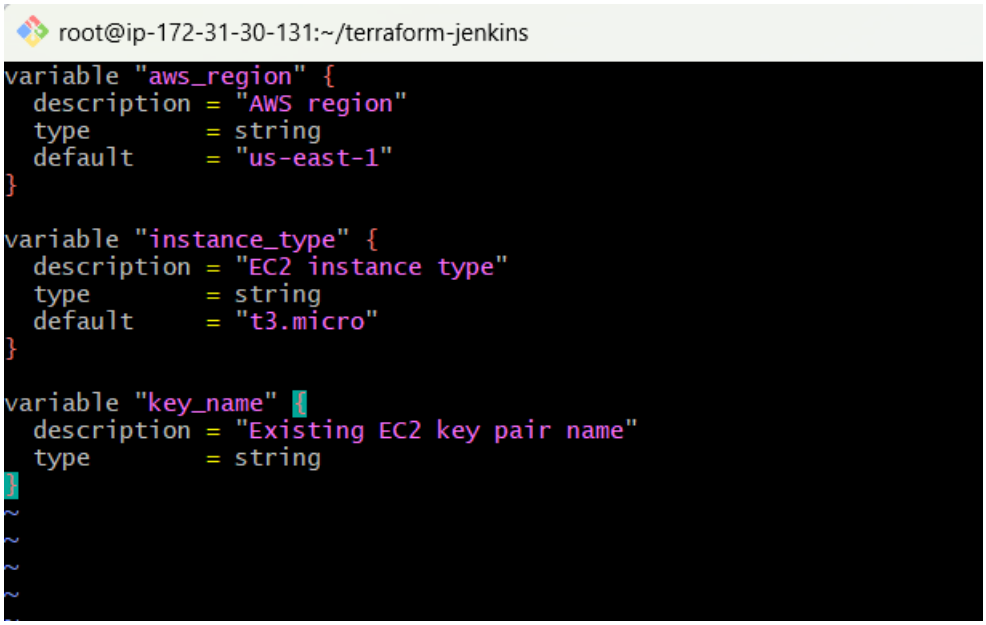
The region is provided through:

var.aws_region

4. Create variables.tf

Create:

vi variables.tf



Here:

aws_region → AWS region

instance_type → EC2 instance type

key_name → EC2 key pai

5. Create userdata.sh

This script will automatically install Jenkins after EC2 is created.

Create:

root@ip-172-31-30-131:~/terraform-jenkins

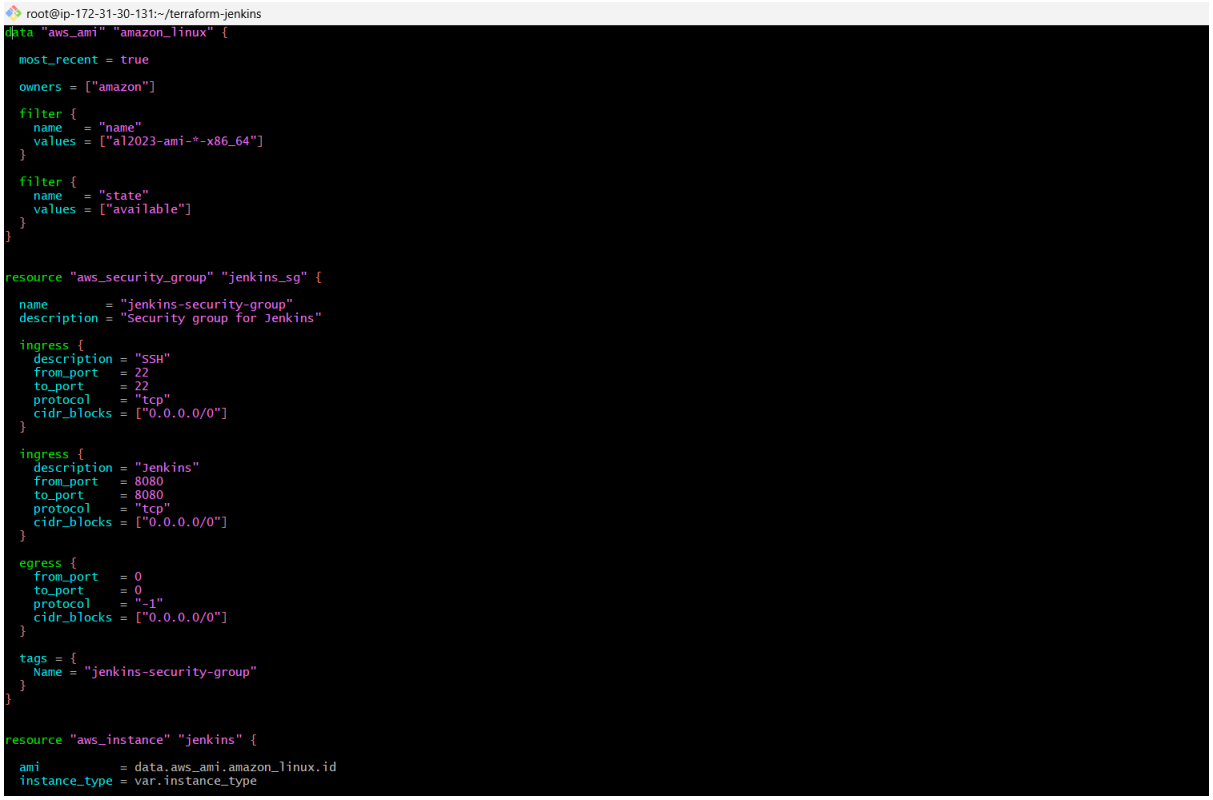
```
#!/bin/bash

set -e

# Update packages
dnf update -y

# Install Java 17
dnf install -y java-17-amazon-corretto

# Add Jenkins repository
curl -fsSL https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key \
    -o /etc/pki/rpm-gpg/jenkins.io-2026.key

cat > /etc/yum.repos.d/jenkins.repo <<'EOF'
[jenkins]
name=Jenkins
baseurl=https://pkg.jenkins.io/redhat-stable
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/jenkins.io-2026.key
EOF

# Install Jenkins
dnf install -y jenkins

# Start Jenkins
systemctl enable jenkins
systemctl start jenkins
~
~
~
~
~
~
~
```

Important

Jenkins repository keys and package URLs can change over time. If the current Jenkins repository rejects the key shown above, use the current installation instructions from the official Jenkins documentation rather than blindly reusing an old key.

6. Create main.tf

Create:



Important: AMI ID

The AMI ID above is **region-specific** and can become outdated.

For your own environment, use an Amazon Linux 2023 AMI available in your selected AWS region.

You can find it from:

AWS Console

↓

EC2

↓

AMIs

↓

Amazon Linux

Or use a Terraform data source instead of hardcoding an AMI ID.

9. Create outputs.tf

Create:

vi outputs.tf

root@ip-172-31-30-131:~/terraform-jenkins

```
output "instance_id" {
  description = "Jenkins EC2 instance ID"
  value       = aws_instance.jenkins.id
}

output "public_ip" {
  description = "Jenkins EC2 public IP"
  value       = aws_instance.jenkins.public_ip
}

output "jenkins_url" {
  description = "Jenkins URL"
  value       = "http://${aws_instance.jenkins.public_ip}:8080"
}
~
~
~
~
~
~
~
~
```

10. Your Final Project Structure

Run:

ls -l

You should have:

terraform-jenkins/

│

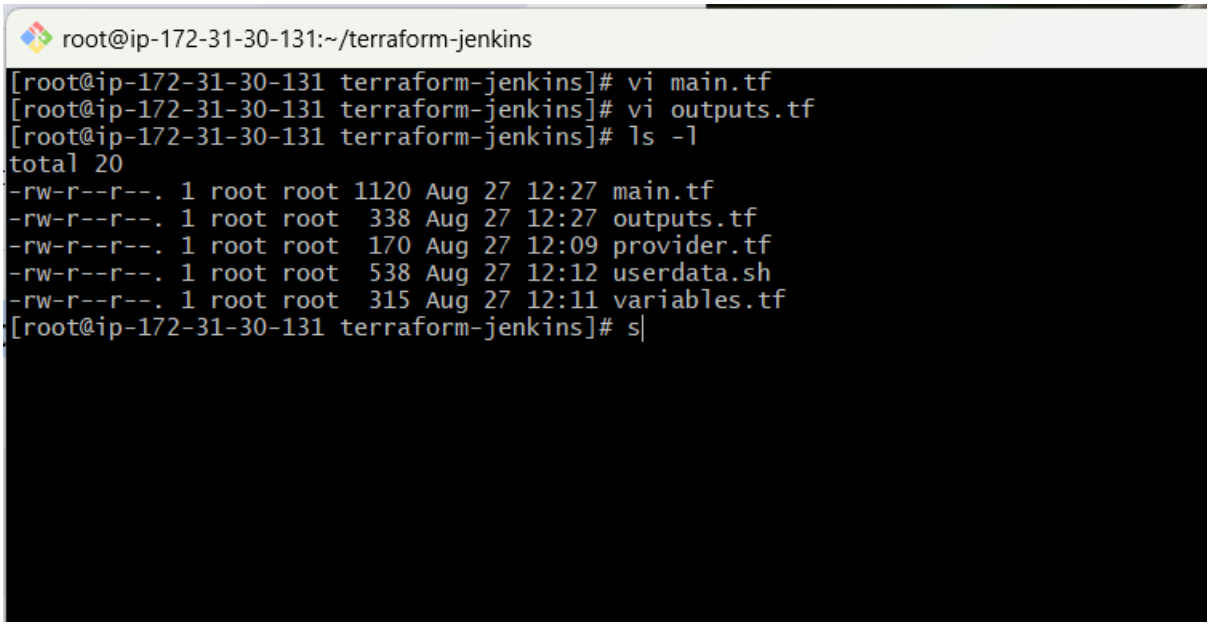
├── main.tf

├── provider.tf

├── variables.tf

├── outputs.tf

└── userdata.sh



11. Initialize Terraform

Run:

terraform init

Expected:

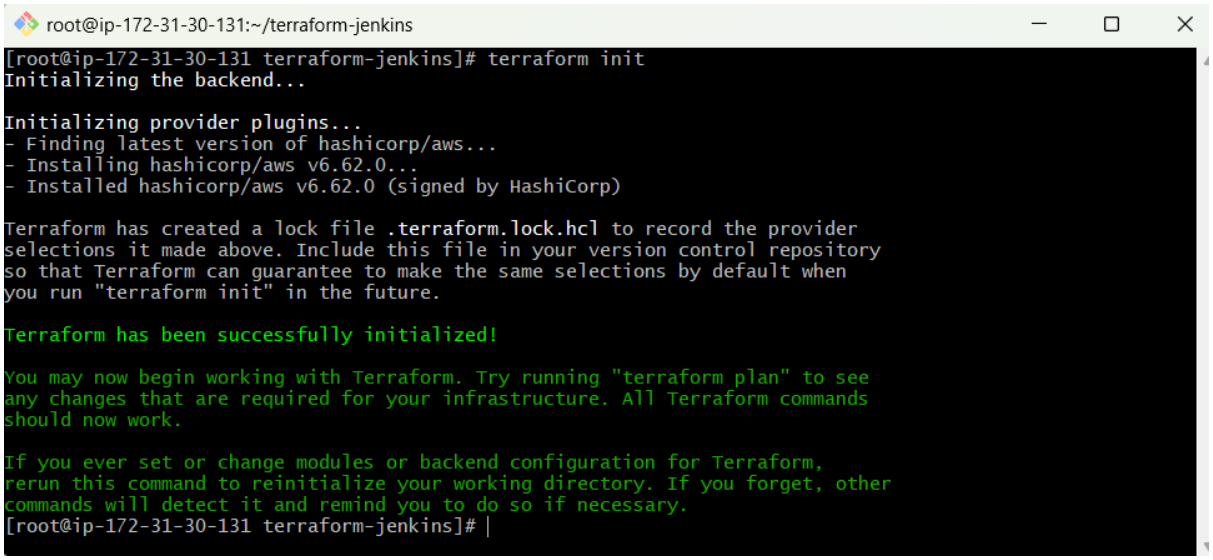
Initializing the backend...

Initializing provider plugins...

Terraform has been successfully initialized!

What does terraform init do?

It downloads the AWS provider and initializes the Terraform working directory



2. Format Terraform Files

Run:

terraform fmt

This formats your .tf files.

---

13. Validate Configuration

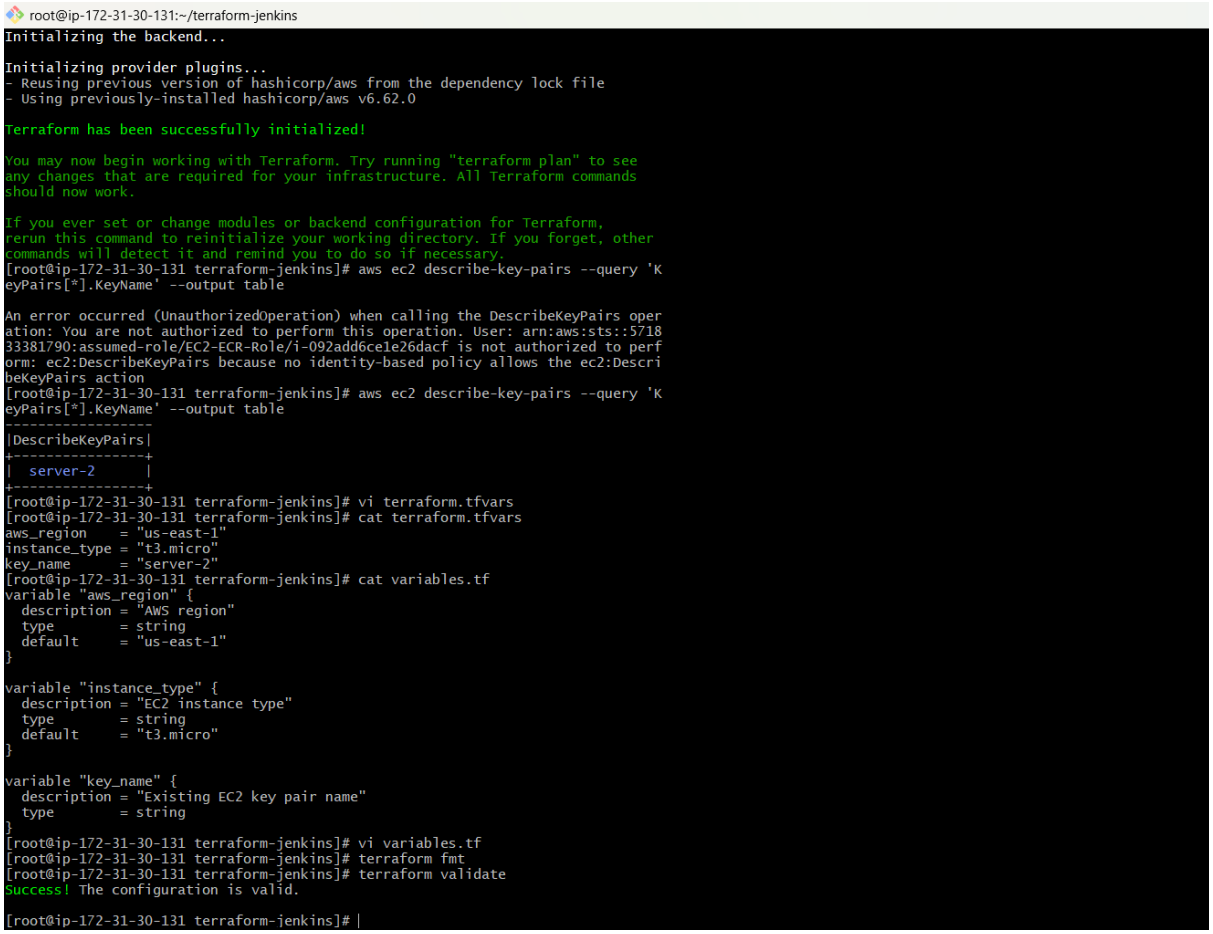
Run:

terraform validate

Expected:

Success! The configuration is valid.

If there is a syntax problem, Terraform will tell you the file and line.



14. Create a Terraform Plan

You need to provide your EC2 key pair name.

For example, if your AWS key pair is:

my-ec2-key

Run:

terraform plan -var="key_name=my-ec2-key"

Terraform will show what it intends to create.

You should see resources similar to:

Plan: 2 to add, 0 to change, 0 to destroy.

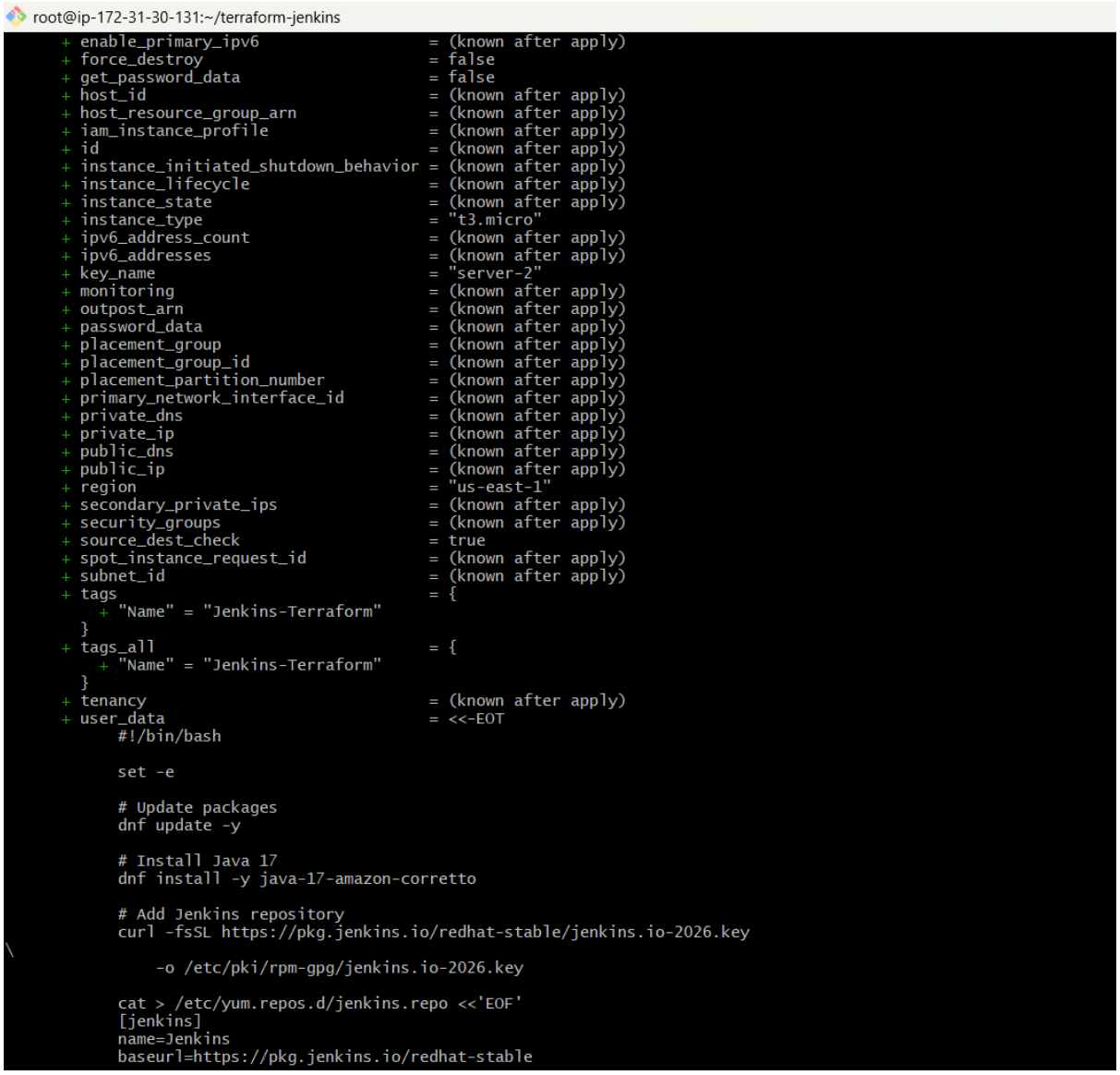
The resources are:

1. Security Group

2. EC2 Instance

root@ip-172-31-30-131:~/terraform-jenkins

```
[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=al2023-ami-*-x86_64" \
  --query 'Images | sort_by(@, &CreationDate)[-1].[ImageId,Name,CreationDate]' \

  --output table
-------------------------------------------------------------
|                      DescribeImages                       |
+-----------------------------------------------------------+
|  ami-0fe74bfcad4fd6bd2                                    |
|  al2023-ami-ecs-hvm-2023.0.20260820-kernel-6.1-x86_64     |
|  2026-08-21T19:35:04.000Z                                 |
+-----------------------------------------------------------+
[root@ip-172-31-30-131 terraform-jenkins]# vi main.tf
[root@ip-172-31-30-131 terraform-jenkins]# vi main.tf
[root@ip-172-31-30-131 terraform-jenkins]# terraform fmt
[root@ip-172-31-30-131 terraform-jenkins]# terraform validate
Success! The configuration is valid.

[root@ip-172-31-30-131 terraform-jenkins]# aws ec2 describe-key-pairs \
  --query 'KeyPairs[*].KeyName' \
  --output table
-----------------
|DescribeKeyPairs|
+----------------+
|  server-2      |
+----------------+
[root@ip-172-31-30-131 terraform-jenkins]# cat terraform.tfvars
aws_region    = "us-east-1"
instance_type = "t3.micro"
key_name      = "server-2"
[root@ip-172-31-30-131 terraform-jenkins]# terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 1s [id=ami-0fe74bfcad4fd6bd2]

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.jenkins will be created
  + resource "aws_instance" "jenkins" {
      + ami                                  = "ami-0fe74bfcad4fd6bd2"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
```



Step 5 — Verify the IAM change

After attaching the EC2 policy, wait a few seconds and run:

aws sts get-caller-identity

Then test whether your role can describe EC2 resources:

aws ec2 describe-security-groups --region us-east-1

If this returns security-group information rather than AccessDenied, that's a good sign.

Step 6 — Run Terraform Plan Again

Go back to:

cd terraform-jenkins

Run:

terraform plan -var="key_name=server-2"

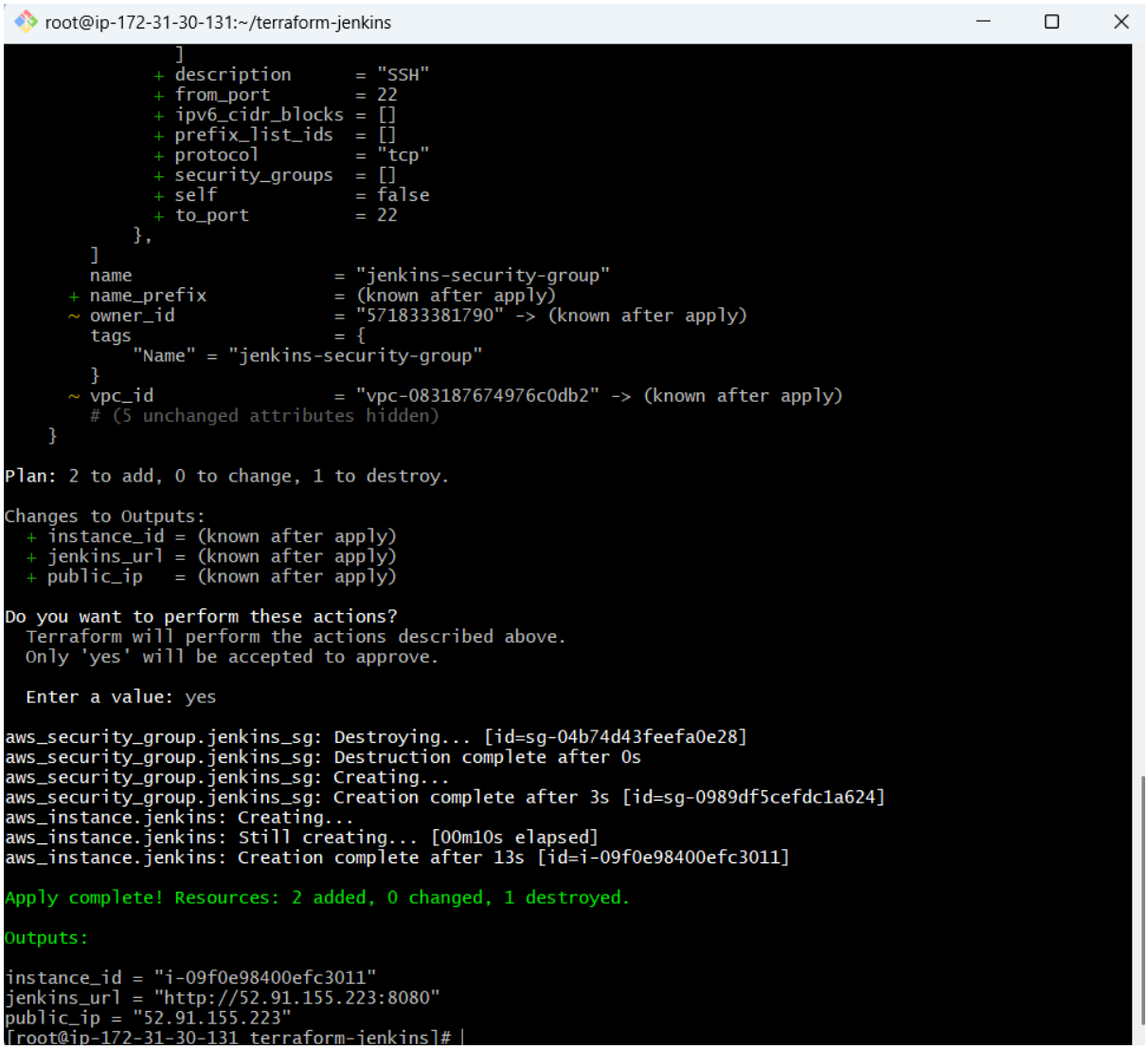
You should see:

Plan: 2 to add, 0 to change, 0 to destroy.

The two resources are:

aws_security_group.jenkins_sg

aws_instance.jenkins



Step 7 — Apply Again

Now run:

terraform apply -var="key_name=server-2"

When prompted:

Do you want to perform these actions?

enter:

yes

This time Terraform should create:

Security Group

↓

EC2 Instance

↓

User Data

↓

Java 17

↓

Jenkins

Step 8 — Check Terraform Output

After successful completion:

terraform output

You should get something similar to:

instance_id = "i-0123456789abcdef"

jenkins_url = "http://54.xx.xx.xx:8080"

public_ip = "54.xx.xx.xx"

You can also run:

terraform output public_ip

and:

terraform output jenkins_url

Step 9 — Check EC2

Use the public IP:

ssh -i server-2.pem ec2-user@<PUBLIC-IP>

For example:

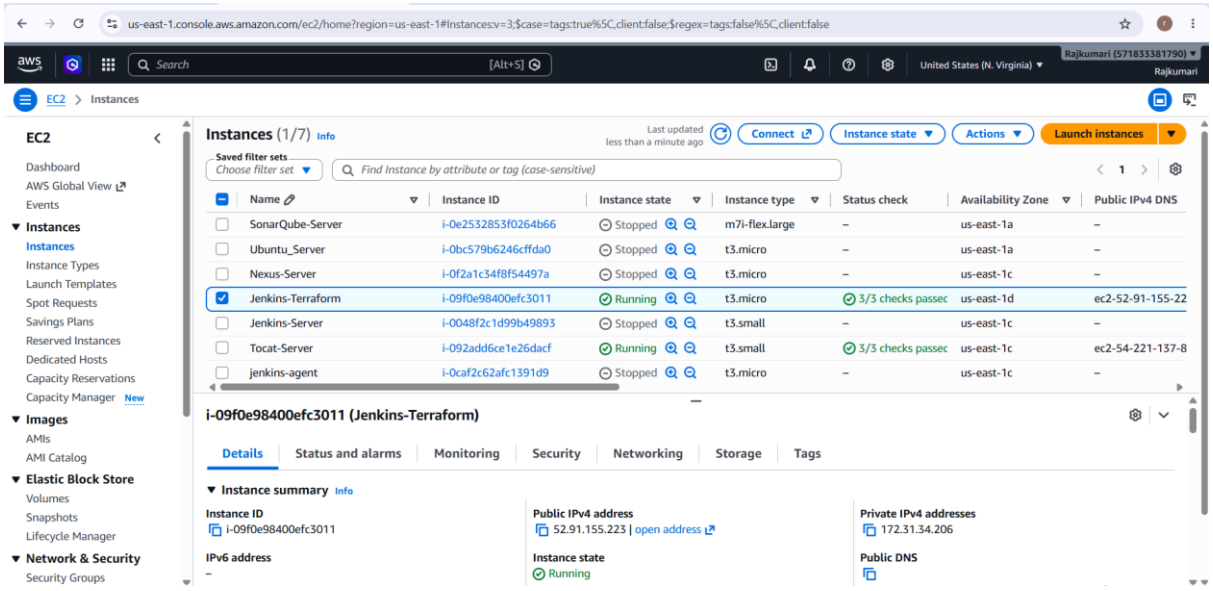
ssh -i server-2.pem ec2-user@54.xx.xx.xx

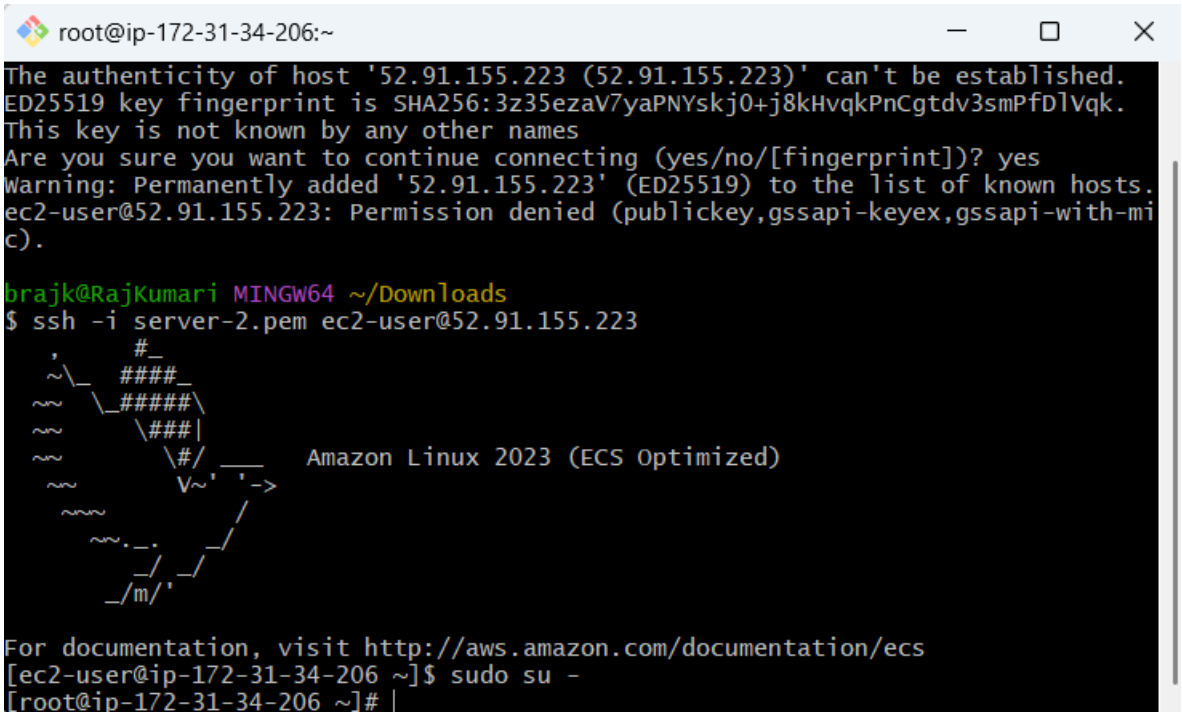
Once inside:

sudo systemctl status jenkins

You want:

Active: active (running)





Step 9 — Check EC2

Use the public IP:

ssh -i server-2.pem ec2-user@<PUBLIC-IP>

For example:

ssh -i server-2.pem ec2-user@54.xx.xx.xx

Once inside:

sudo systemctl status jenkins

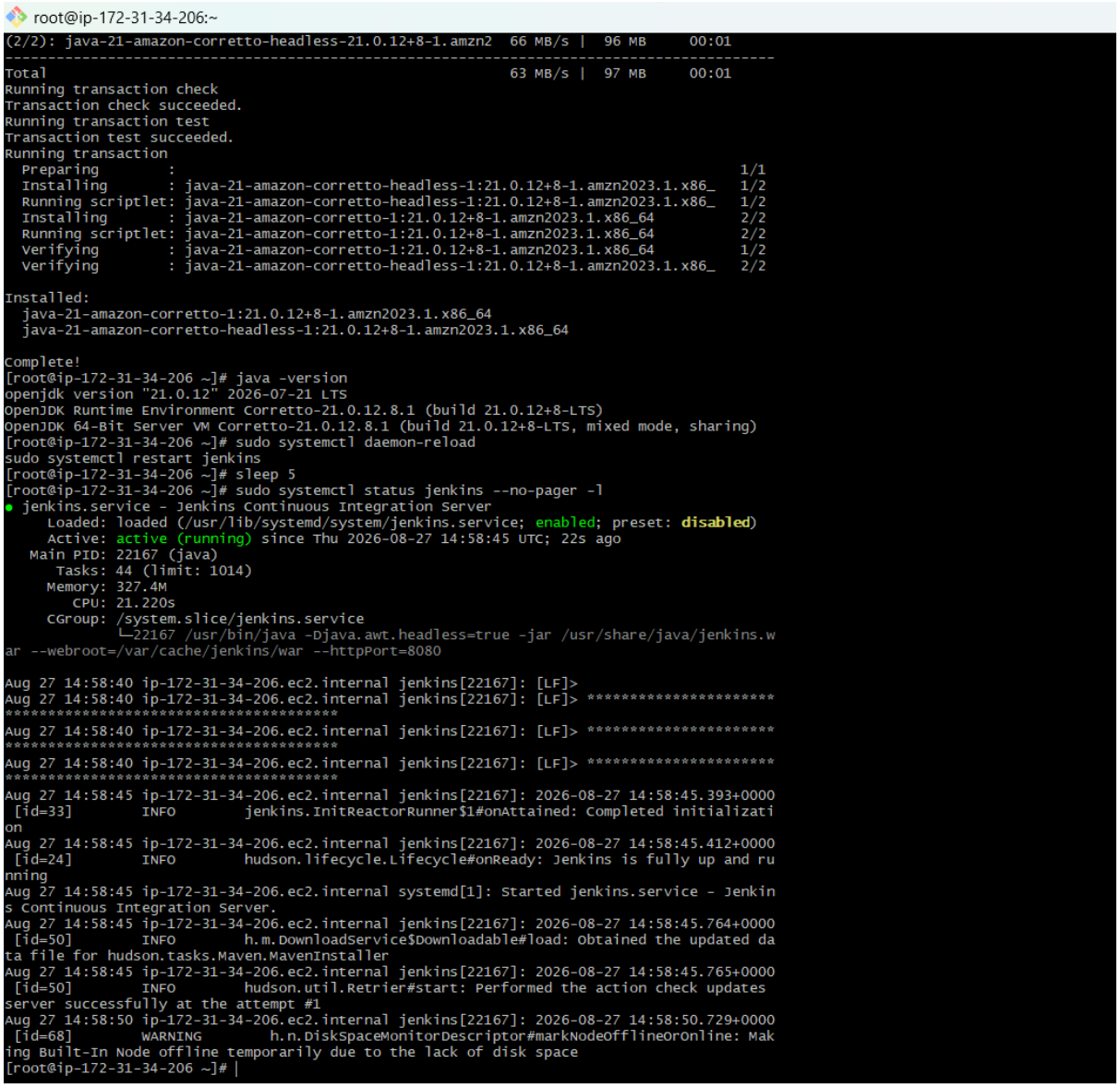
You want:

Active: active (running)

Step 10 — Check Java

Run:

java -version



Step 12 — Get Jenkins Initial Password

Run:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword

Copy the output.

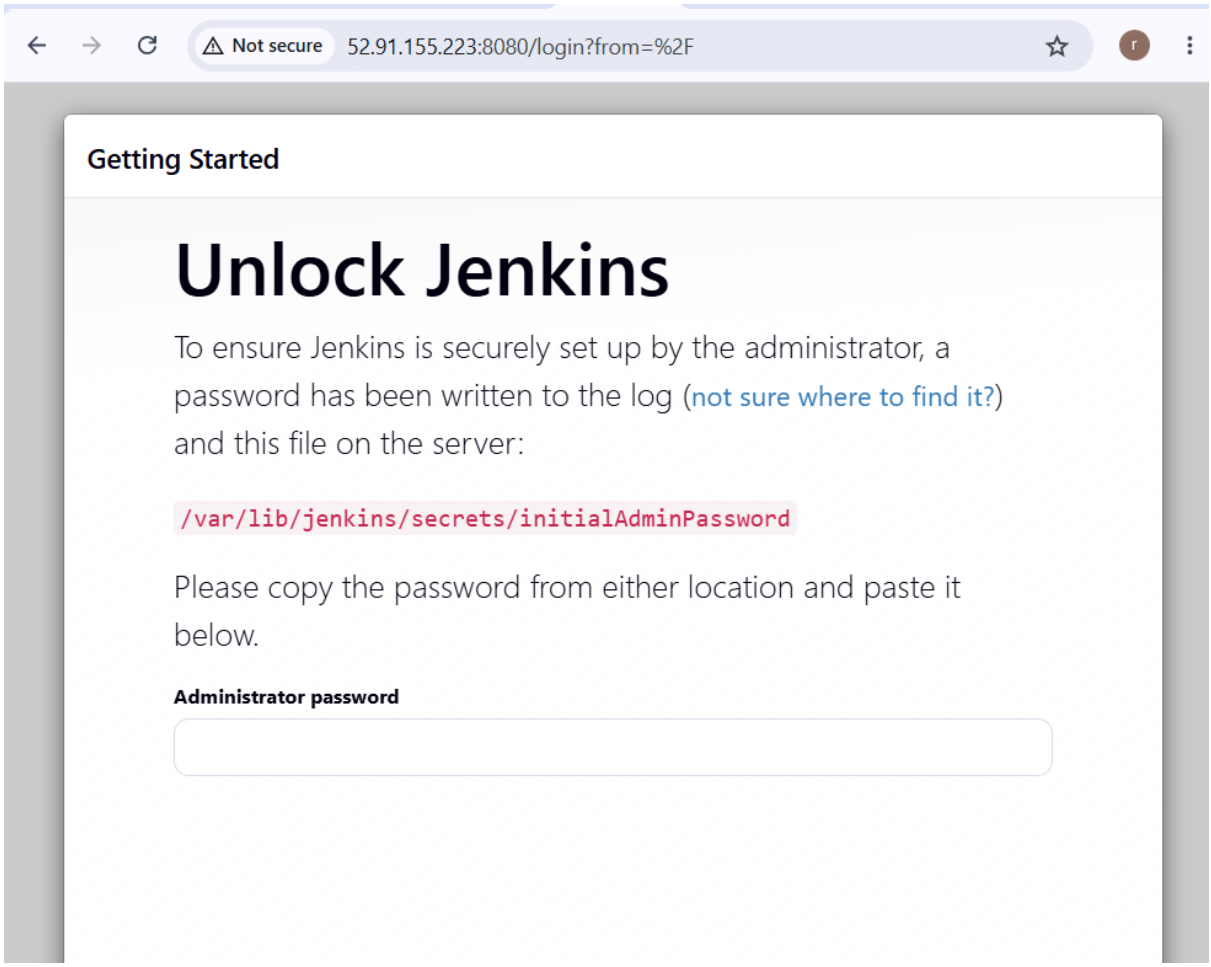
Then open this in your browser:

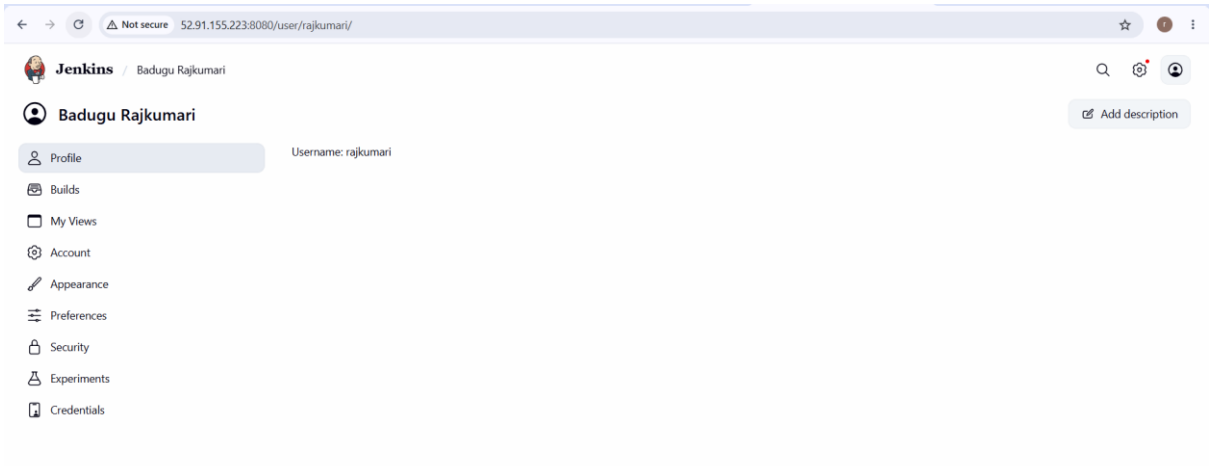
http://<PUBLIC-IP>:8080

For example:

http://54.xx.xx.xx:8080

Enter the initial password.


Not secure
52.91.155.223:8080/login?from=%2F
Getting Started
Unlock Jenkins
To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:
/var/lib/jenkins/secrets/initialAdminPassword
Please copy the password from either location and paste it below.
Administrator password


Not secure
52.91.155.223:8080/user/rajkumari/
Jenkins
Badugu Rajkumari
Badugu Rajkumari
Add description
Profile
Builds
My Views
Account
Appearance
Preferences
Security
Experiments
Credentials
Username: rajkumari

5.Create one Jenkins job to build and push the Docker image to Docker Hub.

Source: https://github.com/betawins/Python-app.git

Source Codes: https://github.com/betawins/docker-tasks.git

- From the**frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.
- From the**Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.
- From the**Node.js-based source code**, write a Dockerfile, build with tagv1, run, and push it to the Docker registry.
- Write a**docker-compose file**to set up WordPress with a MySQL database.

Overview

TASK 1: Jenkins Job → GitHub Python App → Docker Build → Docker Image → Docker Hub Push

TASK 2: Frontend Source → Dockerfile → Docker Build → Docker Run → Docker Hub

TASK 3: Java Source → Dockerfile → Docker Build → Docker Run → Docker Hub

TASK 4: Node.js Source → Dockerfile → Docker Build → Docker Image:v1 → Docker Run → Docker Hub

TASK 5: Docker Compose → WordPress + MySQL

Part 1 — Jenkins Job to Build and Push Docker Image

We will start with the Python application because you specifically provided:

Python-app GitHub Repository

The repository contains:

app.py

requirements.txt

Dockerfile

Jenkinsfile

according to the current repository listing

Step 1 — Verify Docker on Jenkins Server

SSH into your Jenkins EC2:

ssh -i server-2.pem ec2-user@<JENKINS-IP>

Check Docker:

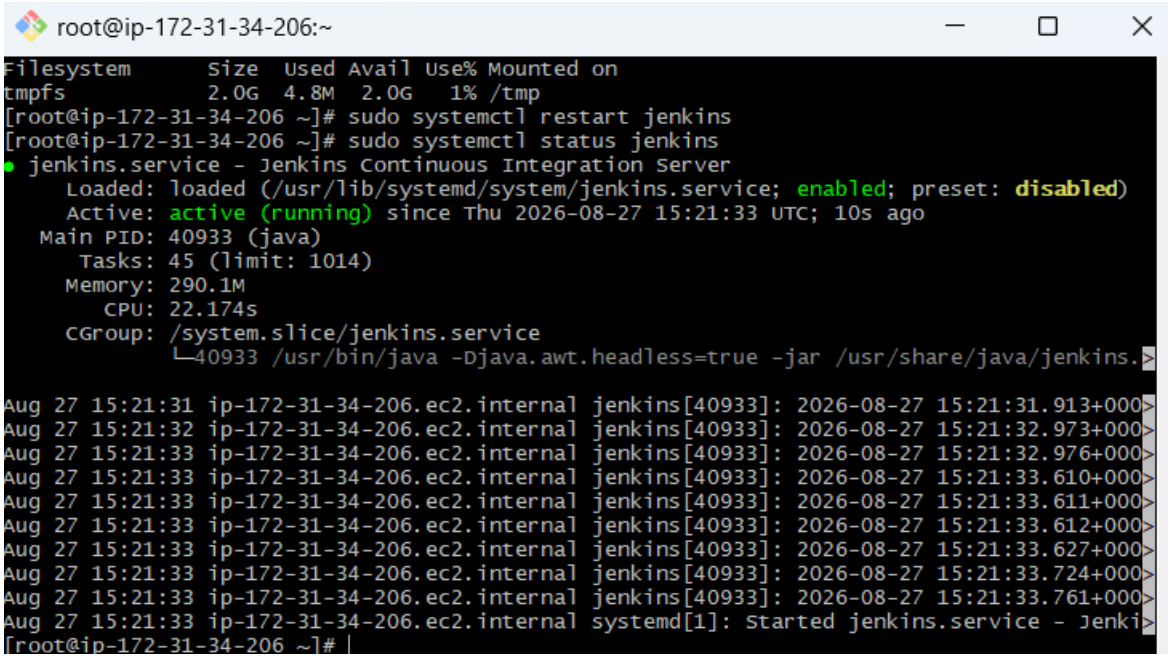
docker --version

Example:

Docker version 28.x

Check Jenkins:

sudo systemctl status jenkins



Step 2 — Add Jenkins User to Docker Group

This is important because Jenkins needs to execute Docker commands.

Run:

sudo usermod -aG docker jenkins

Then restart Jenkins:

sudo systemctl restart jenkins

Verify:

sudo -u jenkins docker ps

If you get:

permission denied

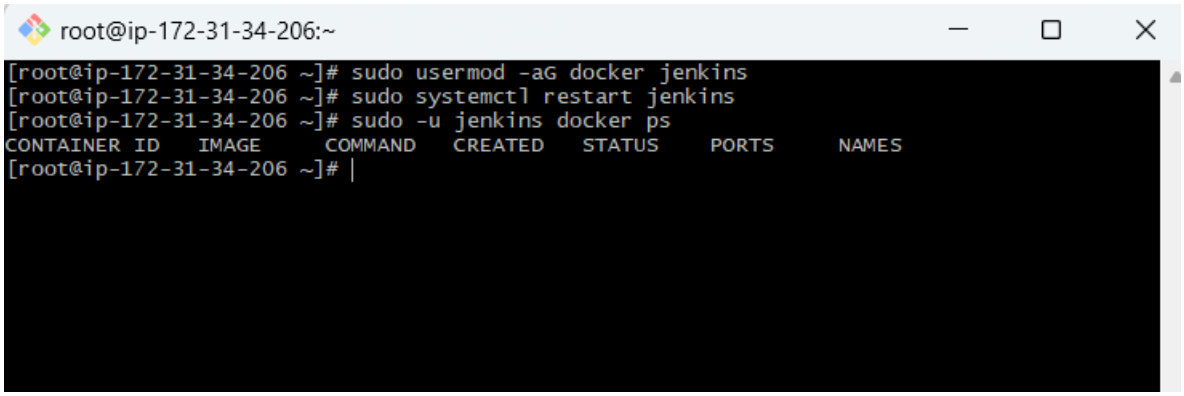
restart Jenkins again or restart the server:

sudo reboot

After reconnecting:

sudo -u jenkins docker ps

It should work.



Step 3 — Create Docker Hub Repository

Go to:

Docker Hub

Create a repository such as:

python-app

Suppose your Docker Hub username is:

rajkumari

Your image name will be:

rajkumari/python-app:latest

Repositories / Create

Using 0 of 1 private repositories. Upgrade your plan to add more

Create repository

Repository Name *

python-app

Short description

A short description to identify your repository. If the repository is public, this description is used to index your content on Docker Hub and in search engines, and is visible to users in search results.

Visibility

Using 0 of 1 private repositories. Upgrade your plan to add more

Public
Appears in Docker Hub search results

Private
Only visible to you

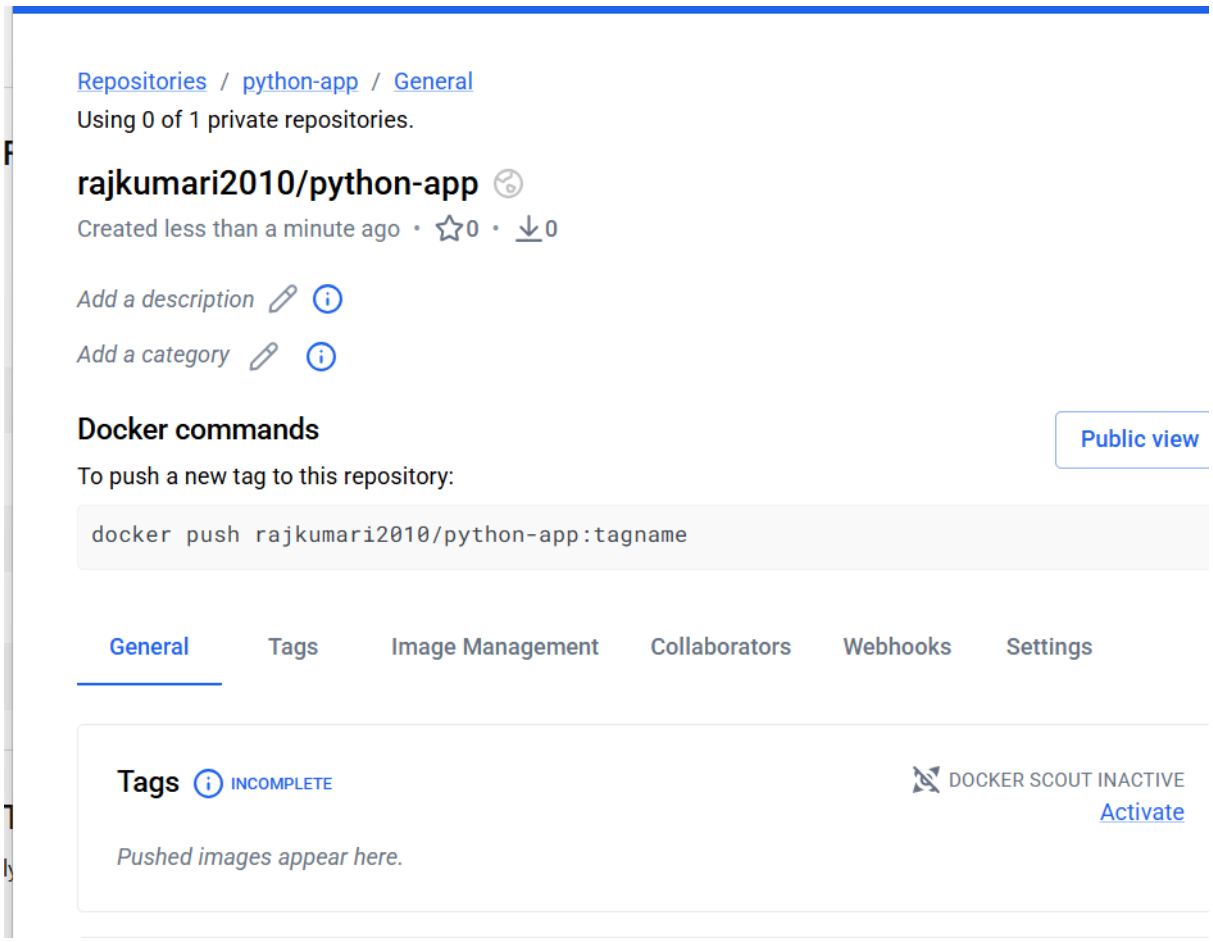
Cancel Create

Pushing images

You can push a new image to this repository using the CLI:

```
docker tag local-image:tagna
me new-repo:tagname
docker push new-repo:tagname
```

Make sure to replace `tagname` with your desired image repository tag.



Step 4 — Create Docker Hub Token

Do **not** put your Docker Hub password directly in Jenkins.

Create a Docker Hub Personal Access Token.

In Docker Hub:

Account Settings

↓

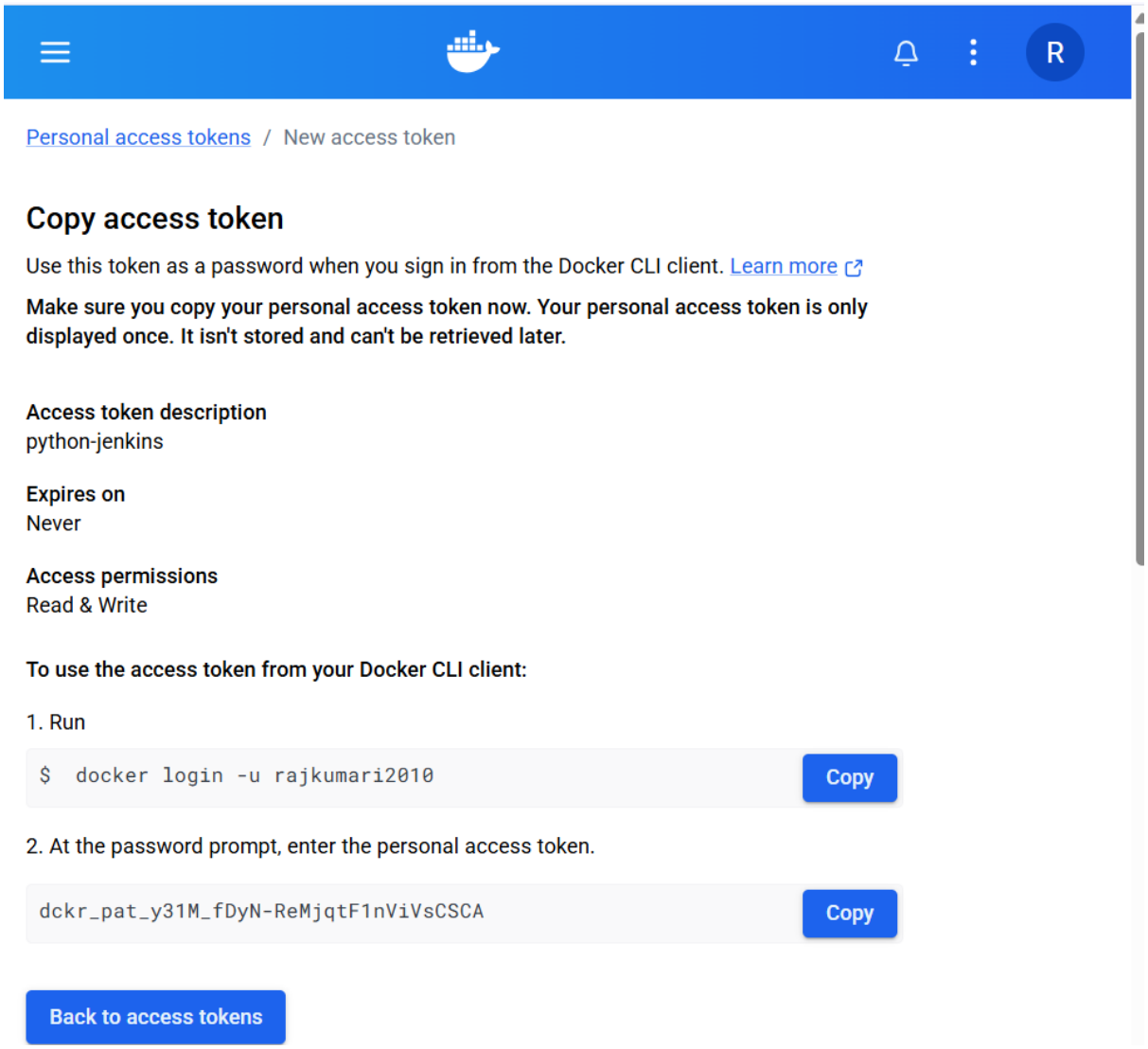
Personal access tokens

↓

Generate new token

Copy the token.

You will use it as a Jenkins credential.



Step 5 — Add Docker Hub Credentials to Jenkins

Go to:

Jenkins

↓

Manage Jenkins

↓

Credentials

↓

System

↓

Global credentials

↓

Add Credentials

Select:

Kind:

Username with password

Enter:

Username:

your Docker Hub username

Password:

Docker Hub PAT

For example:

Username: rajkumari

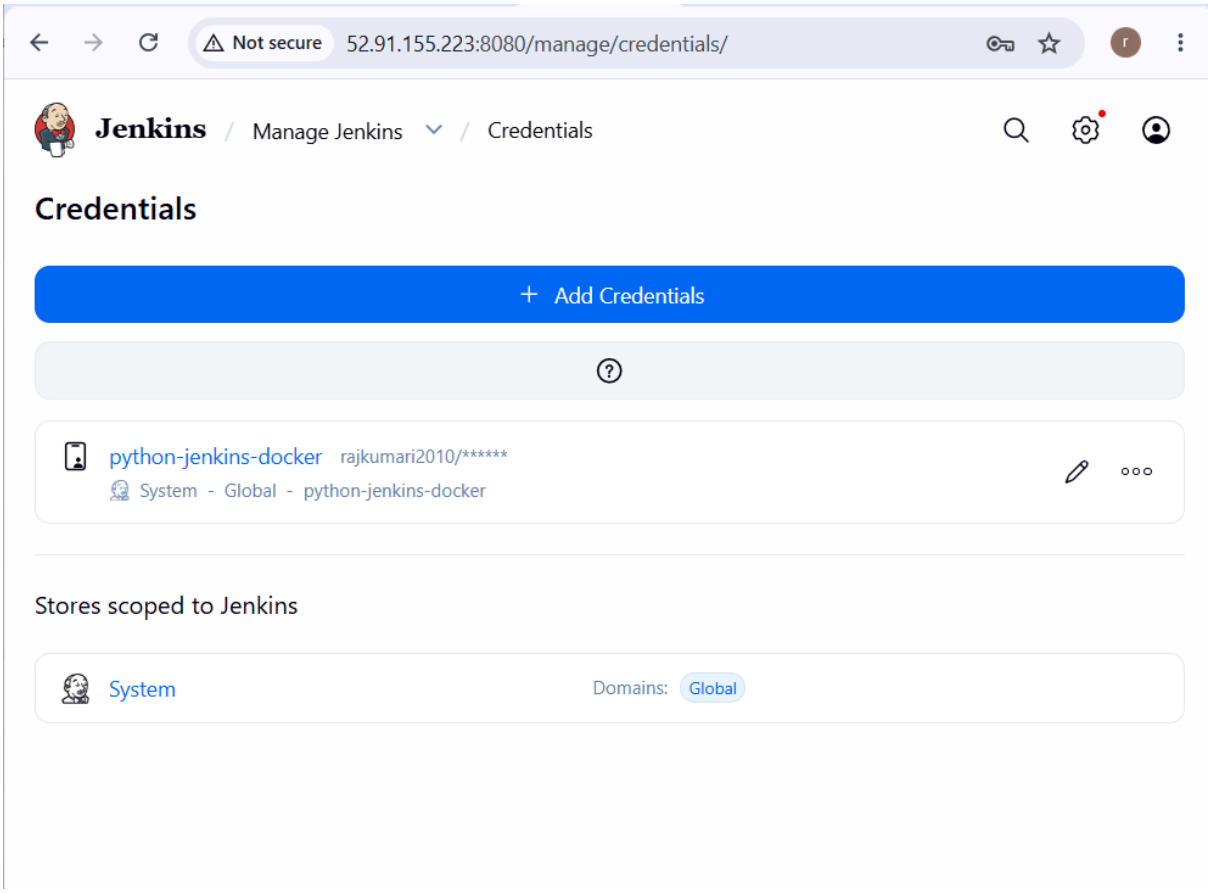
Password: ********

Credential ID:

dockerhub-credentials

Click:

Create



Step 6 — Create Jenkins Pipeline

Go to:

Jenkins Dashboard

↓

New Item

Enter:

python-docker-build-push

Select:

Pipeline

Click:

OK

Step 7 — Add Pipeline

Go to:

Pipeline

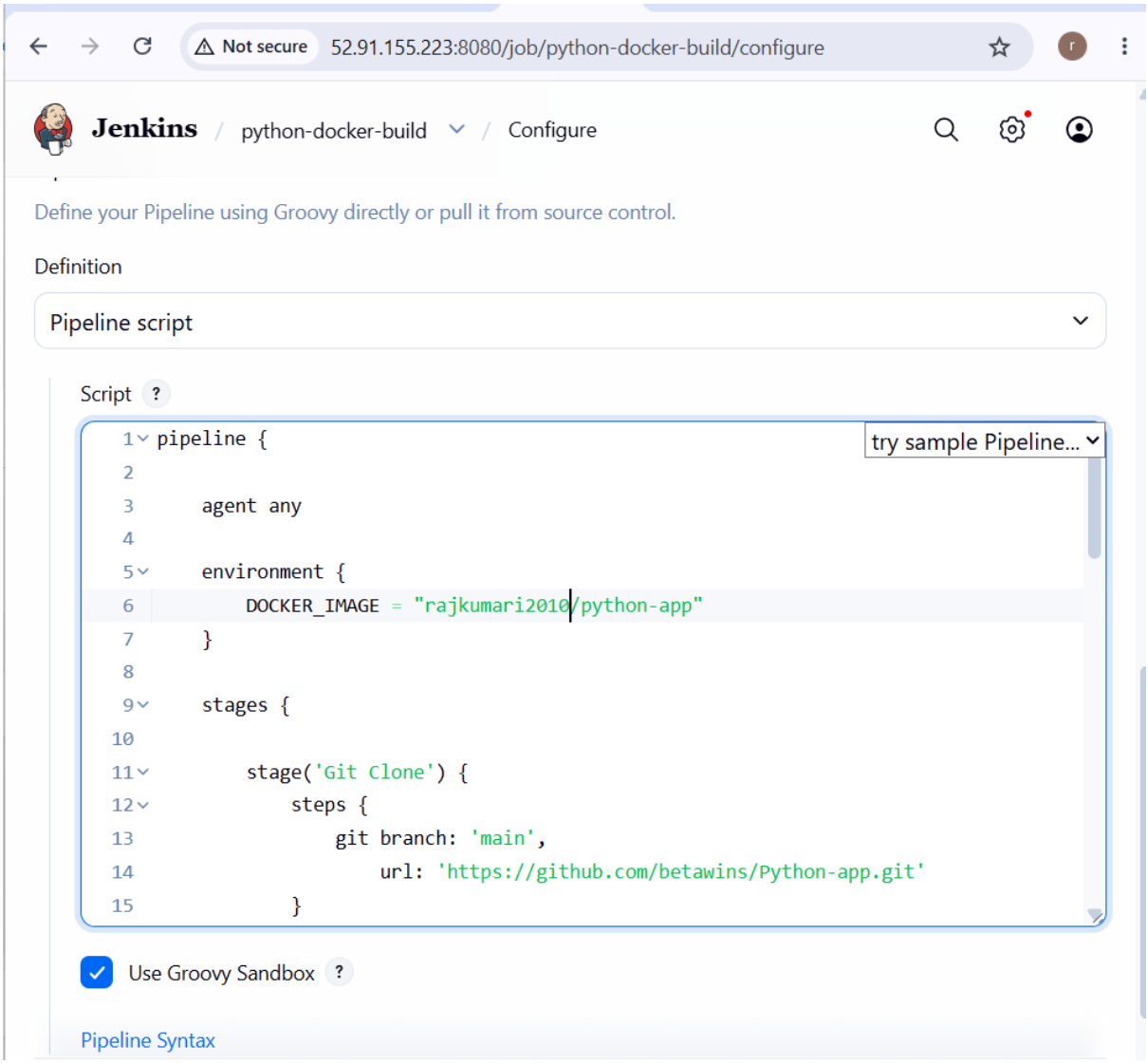
↓

Definition

↓

Pipeline script

Use this pipeline:



Step 8 — Run Jenkins Job

Click:

Build Now

You should see:

Git Clone

↓

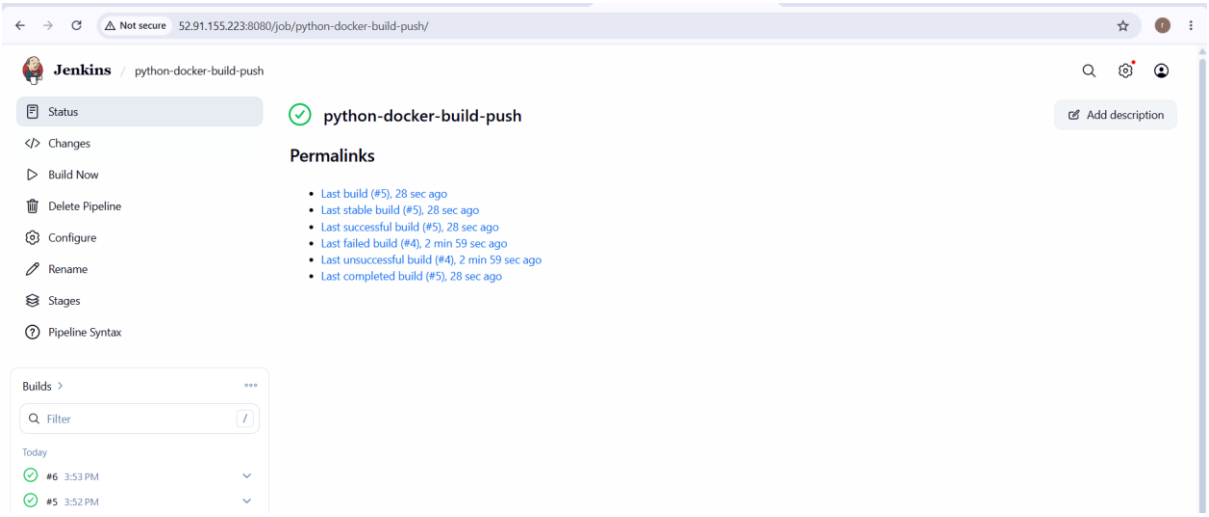
Docker Build

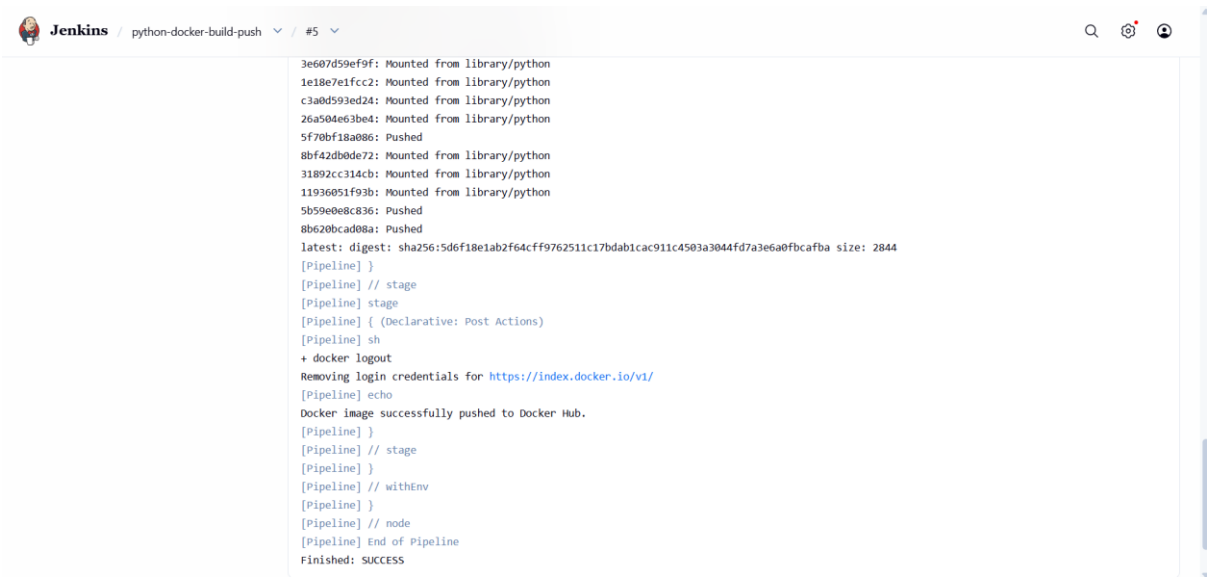
↓

Docker Login

↓

Docker Push





Step 9 — Verify Docker Image

On Jenkins:

docker images

You should see:

rajkumari/python-app latest

In Docker Hub:

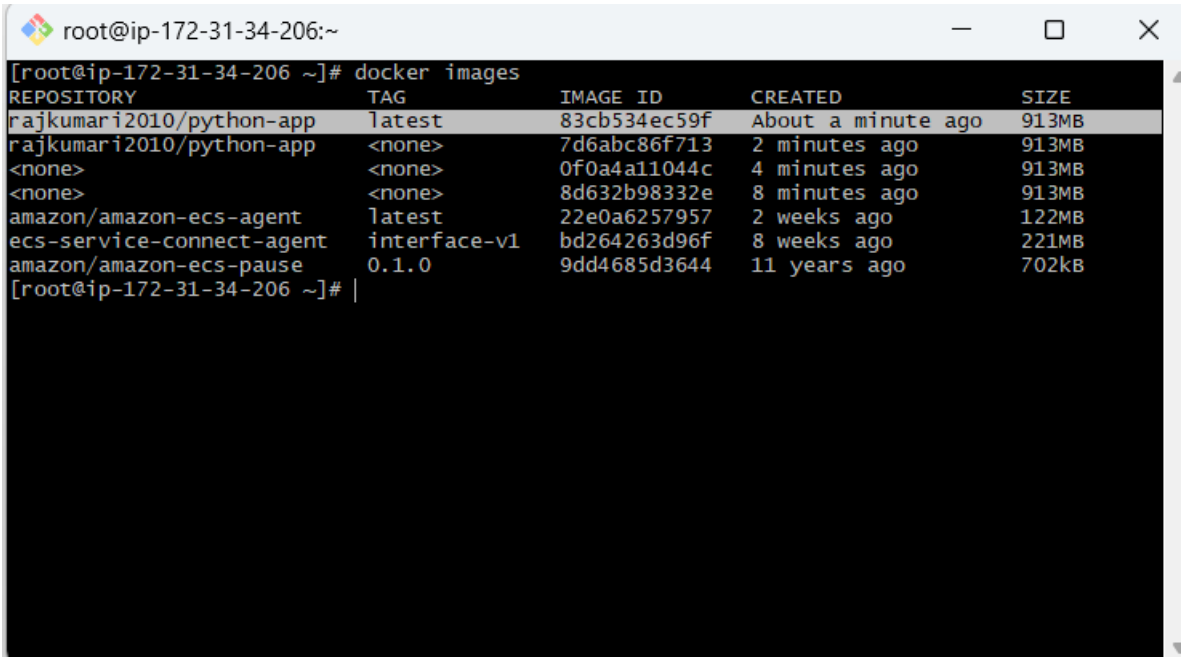
Repositories

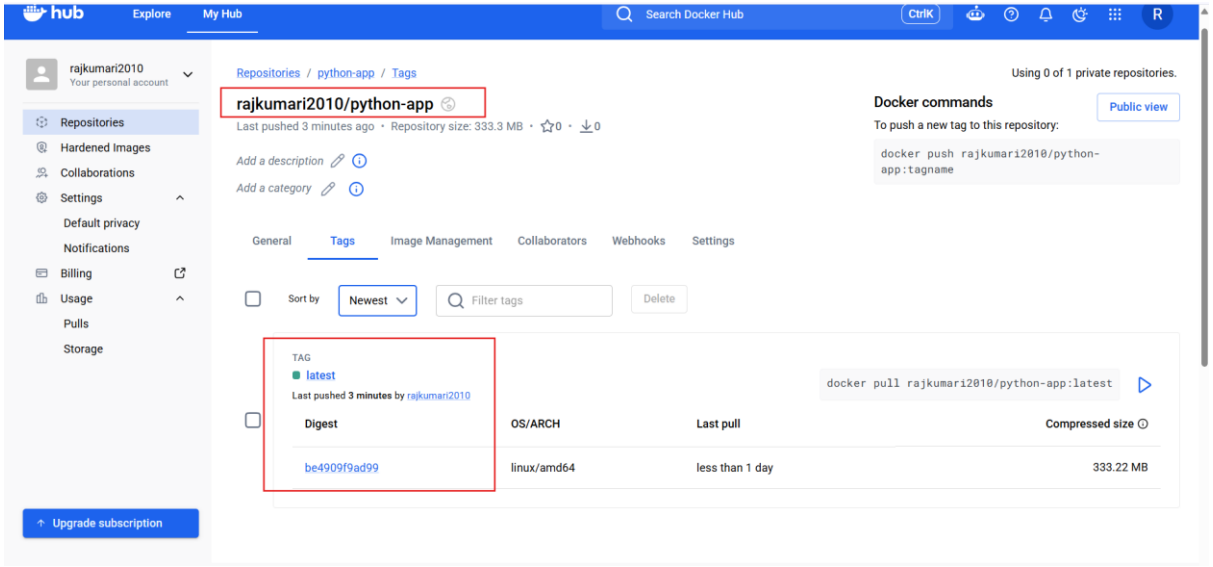
↓

python-app

You should see:

latest





Part 2 — Frontend Source Code

Now complete the frontend assignment.

First clone the source repository:

git clone https://github.com/betawins/docker-tasks.git

Then:

cd docker-tasks

Check the files:

ls -la

Because the repository structure can change, **don't blindly create a Dockerfile in the repository root until you identify the frontend directory**.

Run:

find . -maxdepth 3 -type f | sort

Look for files such as:

package.json

index.html

src/

public/

or:

pom.xml

src/

or:

requirements.txt

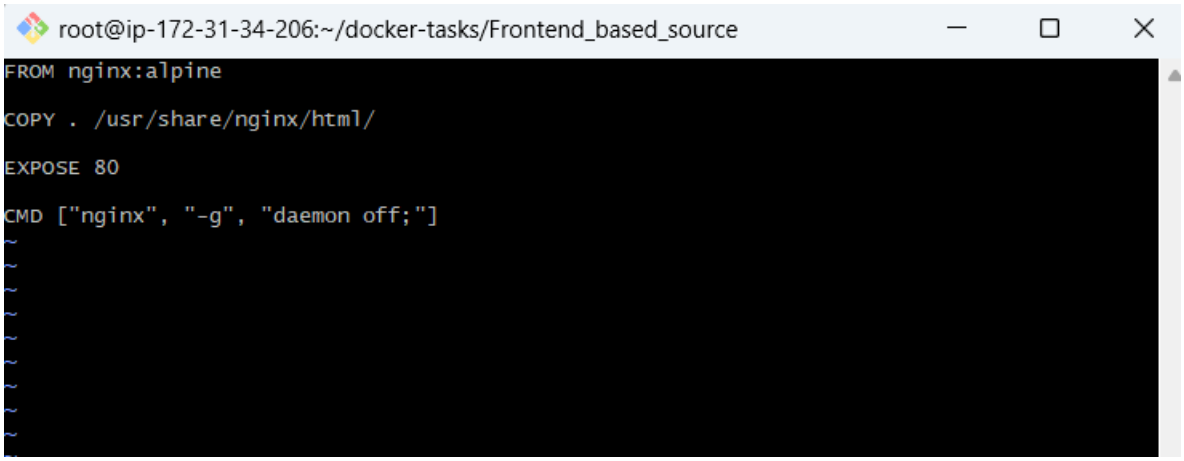
This tells you which application you are working with.

Step 10 — Frontend Dockerfile

If the frontend is a React/Node frontend, a typical production Dockerfile is:

```
FROM node:20-alpine AS build

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build



FROM nginx:alpine

COPY --from=build /app/build /usr/share/nginx/html

EXPOSE 80
```

CMD ["nginx", "-g", "daemon off;"]

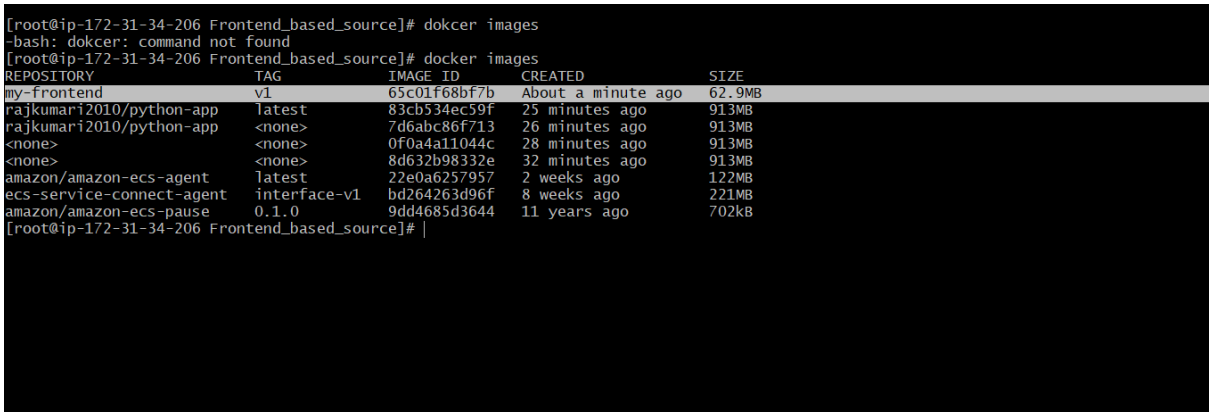


Step 11 — Build Frontend Image

Example:

docker build -t my-frontend:v1 .

docker images



Step 12 — Run Frontend Container

docker run -d \

--name frontend-container \

-p 8081:80 \

my-frontend:v1

Check:

docker ps

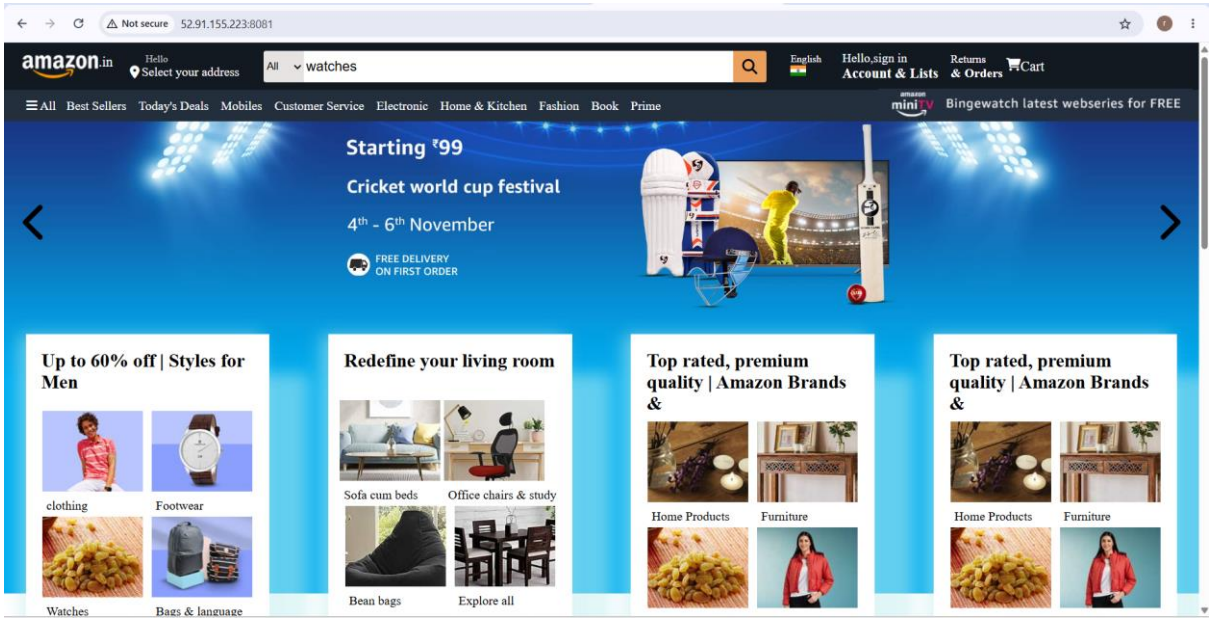
Test:

curl http://localhost:8081

Or open:

http://<SERVER-IP>:8081

Remember to allow TCP 8081 in the EC2 security group if you're accessing it externally.



Step 13 — Tag Frontend Image

Suppose your Docker Hub username is:

rajkumari

Run:

docker tag my-frontend:v1 \

rajkumari/my-frontend:v1

Step 14 — Login to Docker Hub

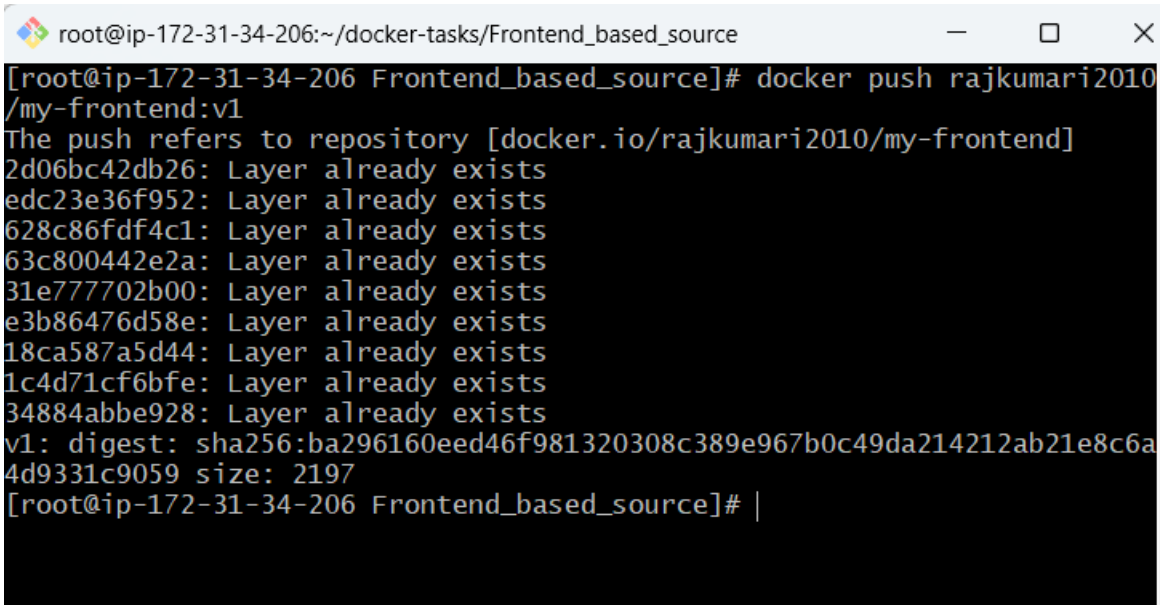
docker login

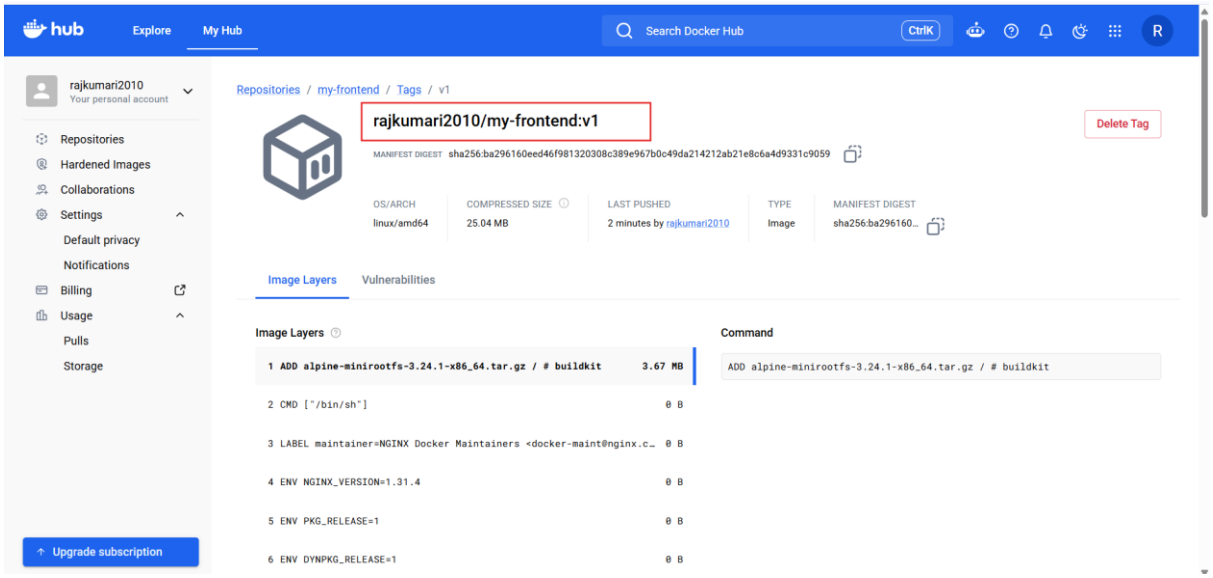
Enter your Docker Hub username and PAT.

Step 15 — Push Frontend Image

docker push rajkumari/my-frontend:v1

Verify in Docker Hub.





Part 3 — Java Application

Now identify the Java source code in docker-tasks.

Look for:

pom.xml

or:

build.gradle

Step 16 — Build Java Image

Go into the Java application directory:

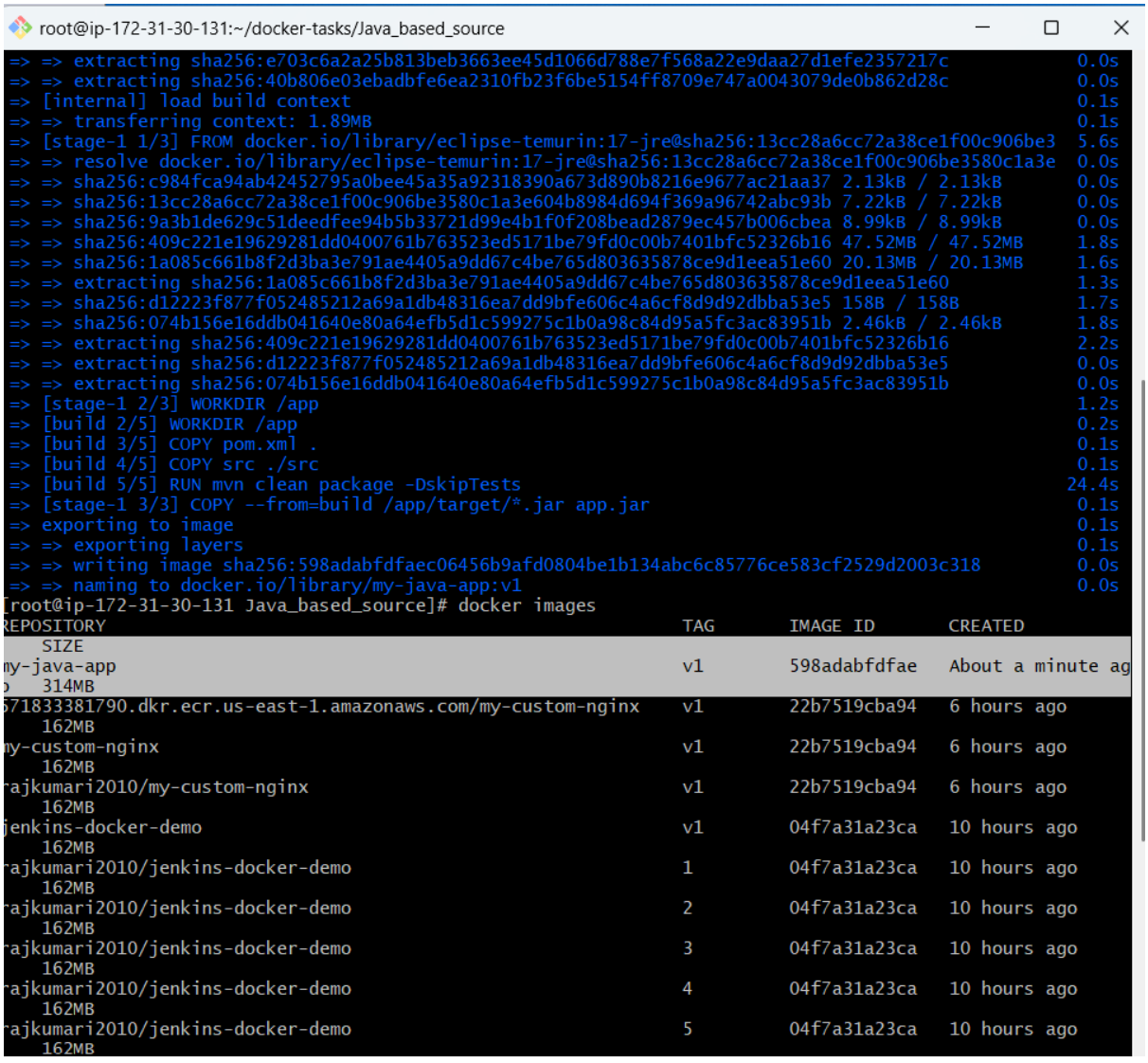
cd <java-directory>

Build:

docker build -t my-java-app:v1 .

Check:

docker images



Step 17 — Run Java Container

```
docker run -d \
  --name java-container \
  -p 8082:8080 \
  my-java-app:v1
```

Check:

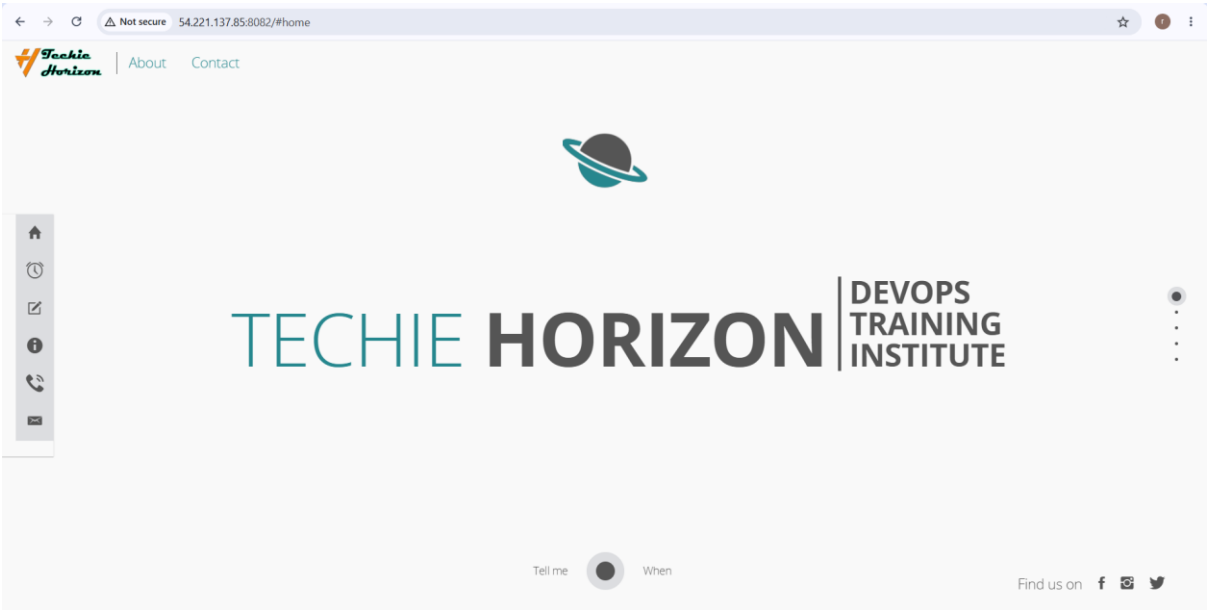
```
docker ps
```

Check logs:

```
docker logs java-container
```

Test:

```
curl http://localhost:8082
```

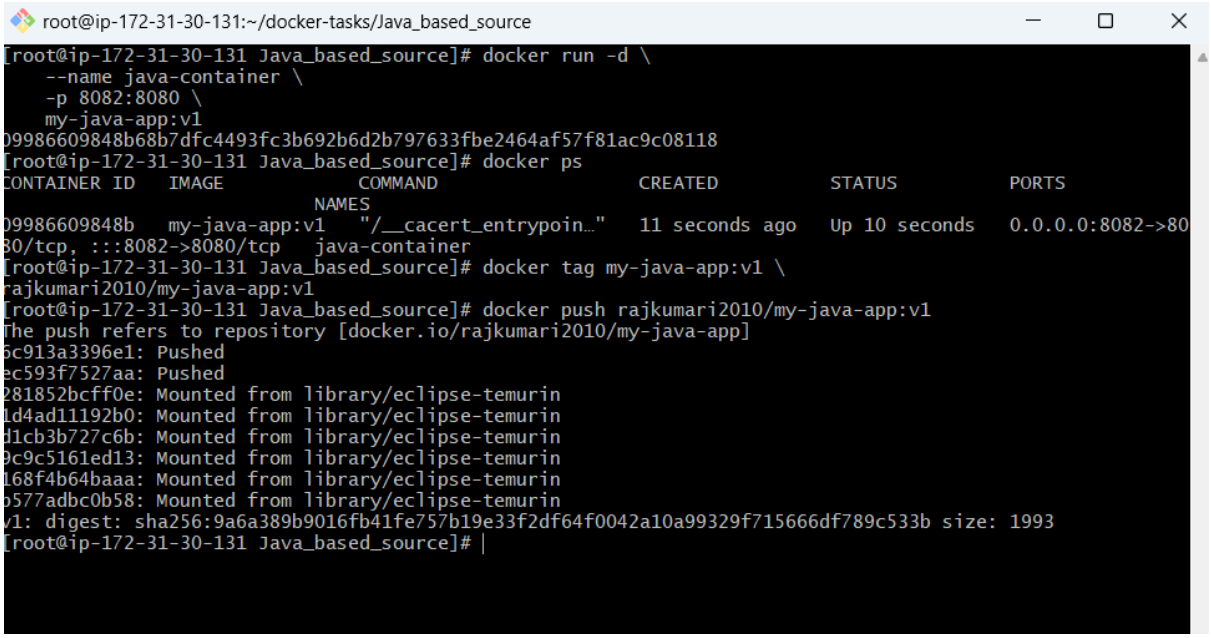


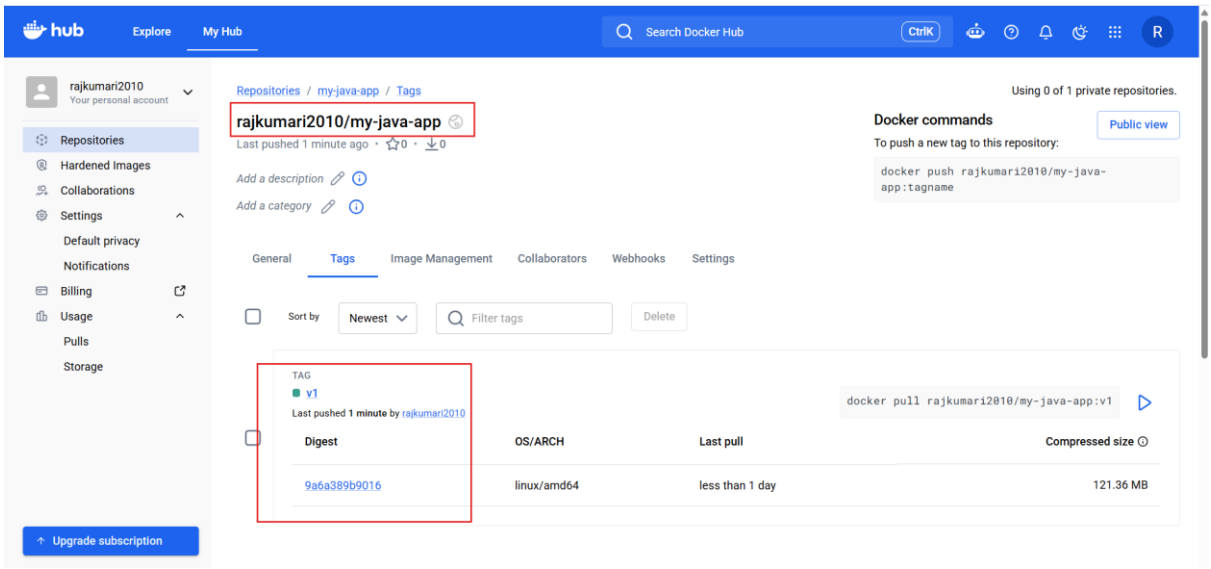
tep 18 — Tag Java Image

```
docker tag my-java-app:v1 \
rajkumari/my-java-app:v1
```

Push:

```
docker push rajkumari/my-java-app:v1
```





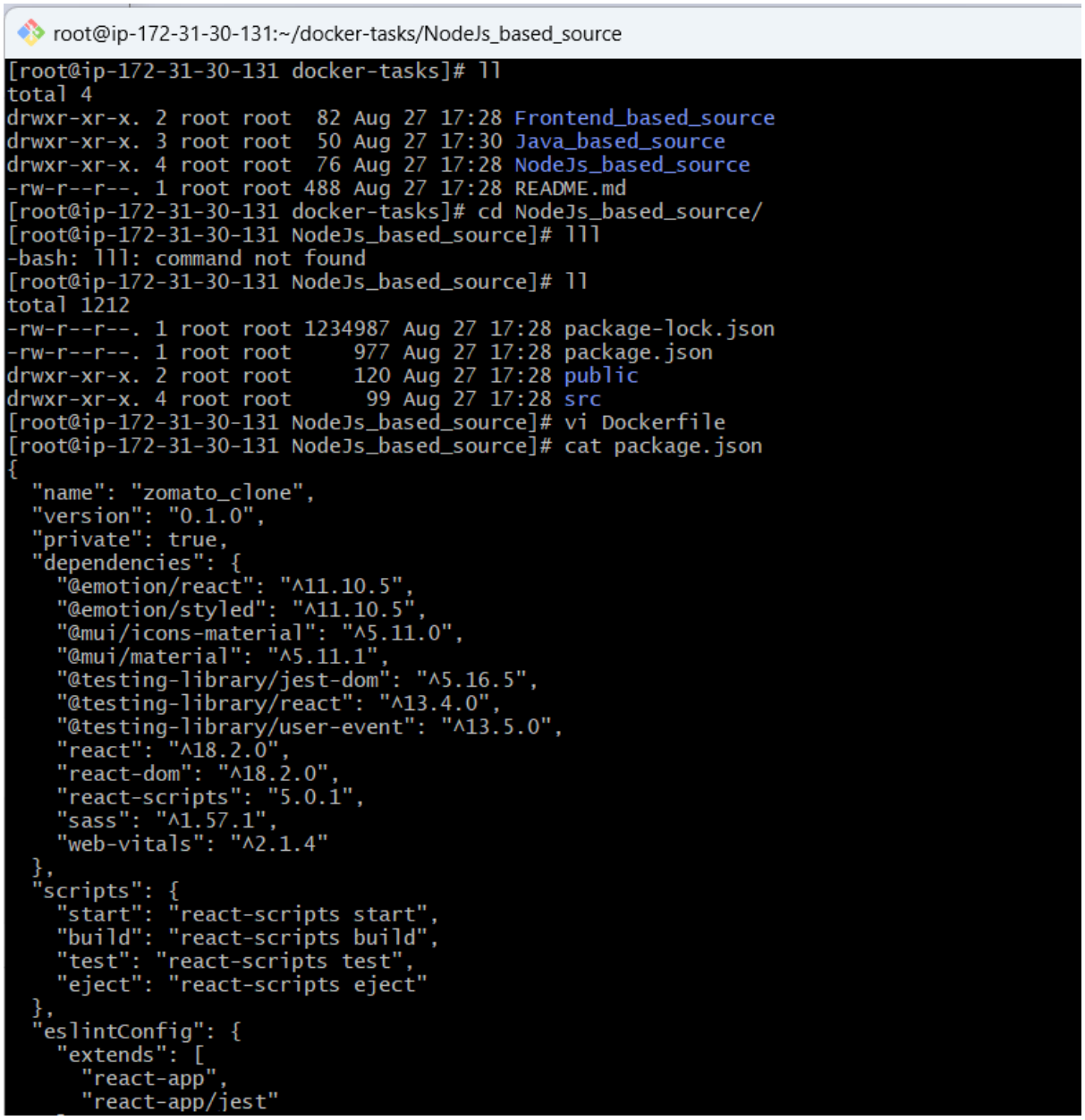
Part 4 — Node.js Application

Find the Node application:

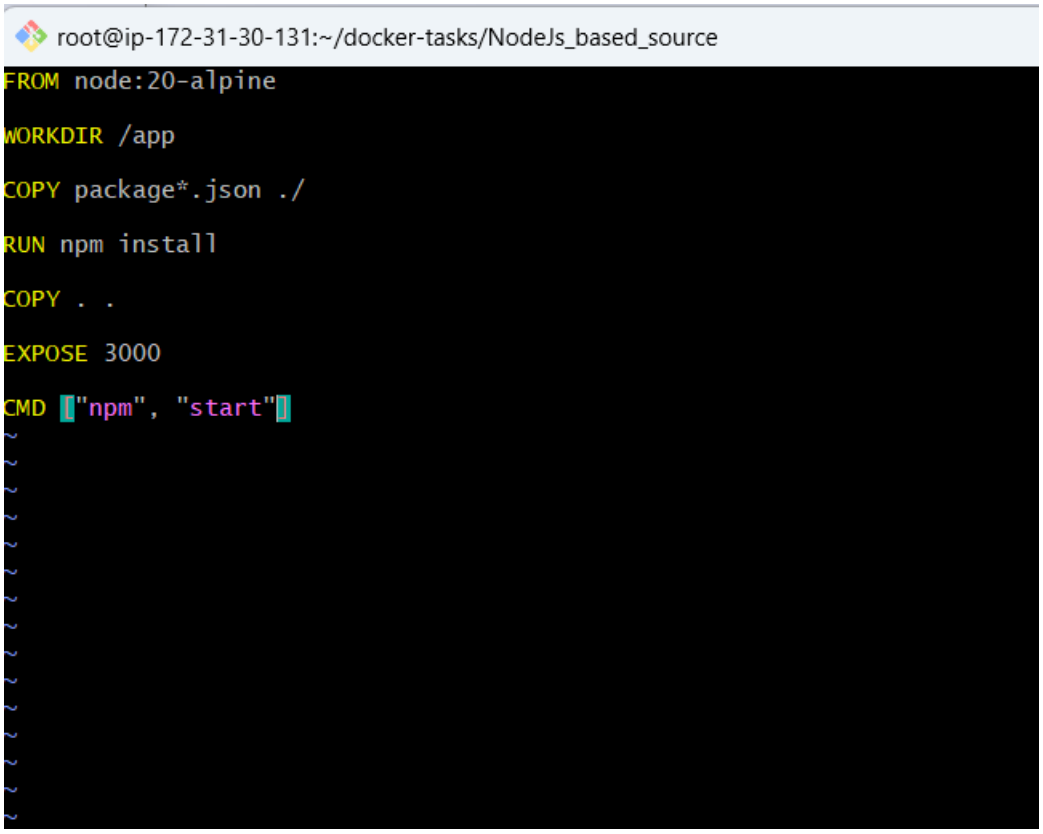
find . -name package.json

Go into its directory:

cd <node-directory>



Vi Dockerfile



Step 19 — Build Node Image With v1

Your assignment specifically says:

Build with tag v1

Run:

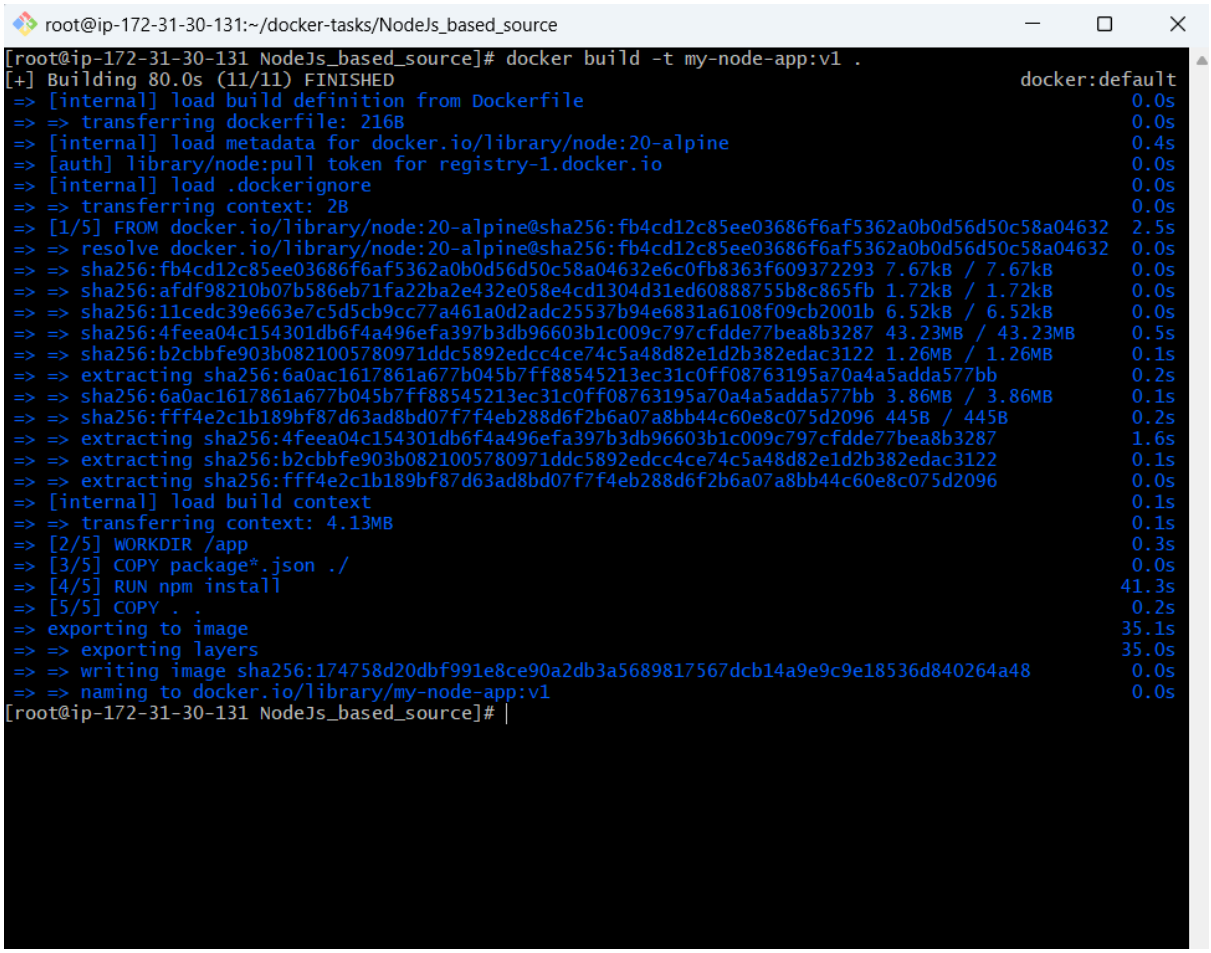
docker build -t my-node-app:v1 .

Verify:

docker images

You should see:

my-node-app v1



Step 20 — Run Node Application

If the application listens on port 3000:

```
docker run -d \
  --name node-container \
  -p 3000:3000 \
  my-node-app:v1
```

Check:

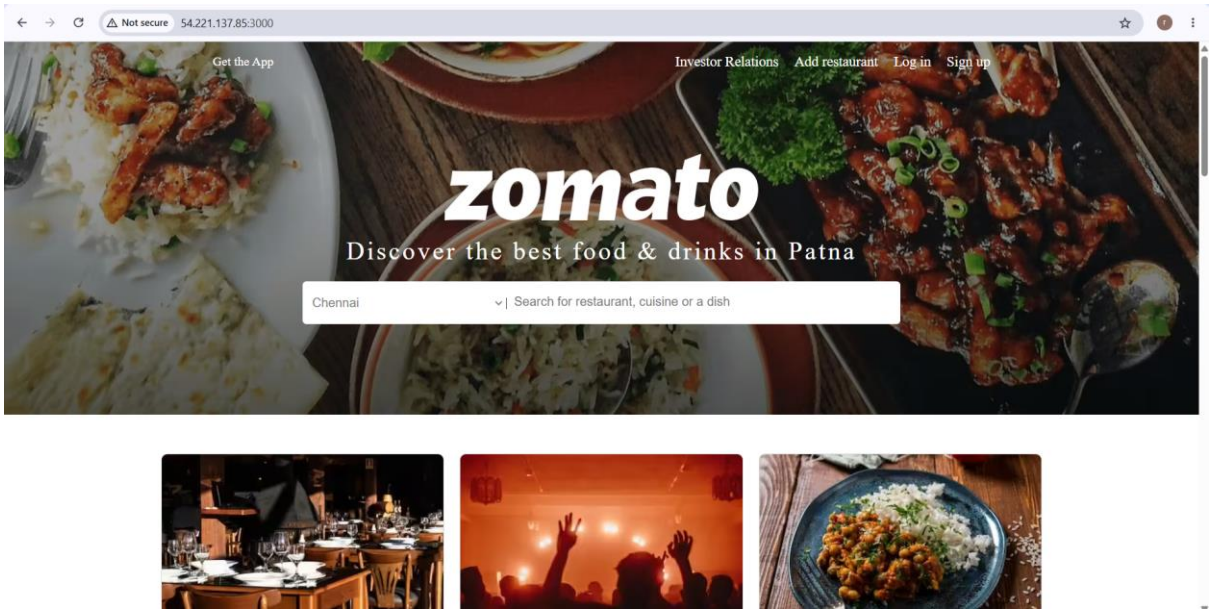
```
docker ps
```

Logs:

```
docker logs node-container
```

Test:

```
curl http://localhost:3000
```



Step 21 — Tag Node Image for Docker Hub

docker tag my-node-app:v1 \

rajkumari/my-node-app:v1

Step 22 — Push Node Image

docker push rajkumari/my-node-app:v1

Now Docker Hub should contain:

rajkumari/

│

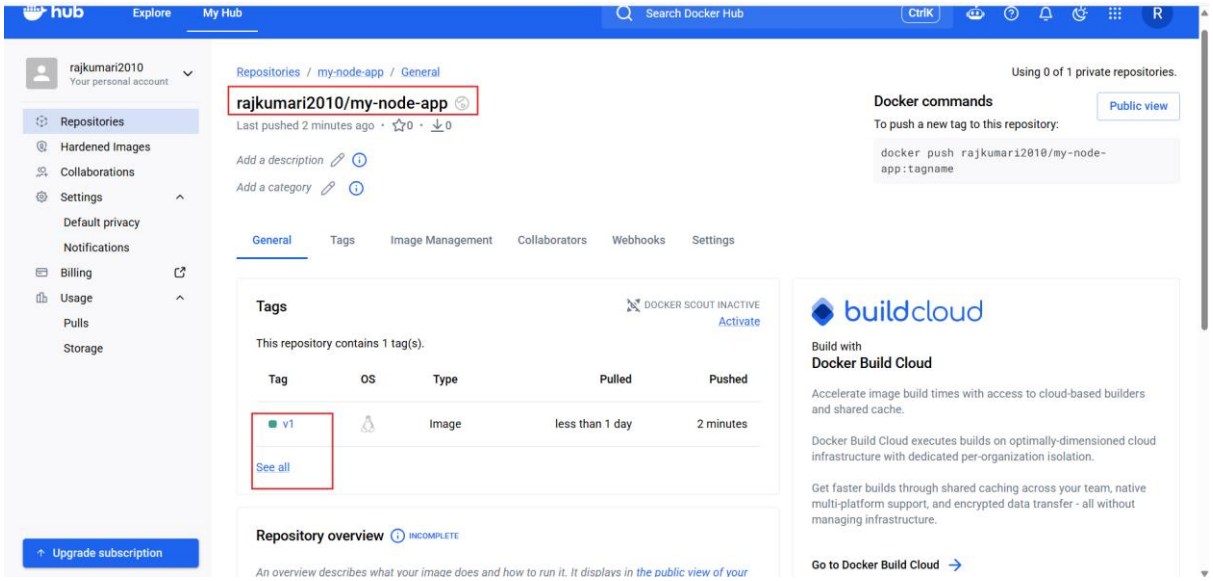
├── python-app:latest

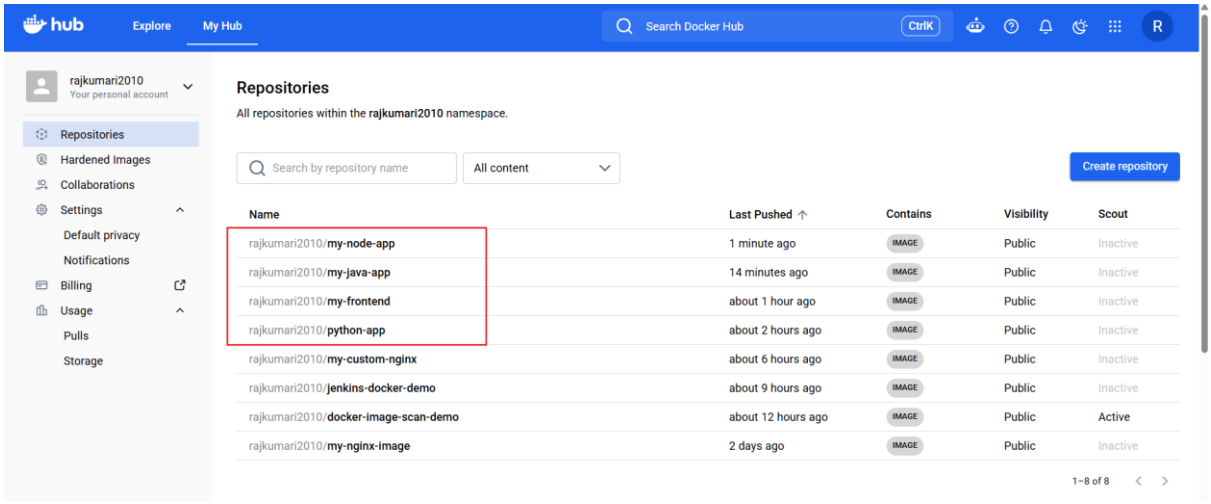
├── my-frontend:v1

├── my-java-app:v1

└── my-node-app:v1

```
root@ip-172-31-30-131:~/docker-tasks/NodeJs_based_source
[root@ip-172-31-30-131 NodeJs_based_source]# docker run -d \
    --name node-container \
    -p 3000:3000 \
    my-node-app:v1
d194d67b66760c7928af72ef95b73d099446f131436c9958783252e757a7f2a2
[root@ip-172-31-30-131 NodeJs_based_source]# docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED          STATUS          PORTS
                    NAMES
d194d67b6676   my-node-app:v1   "docker-entrypoint.s…"   8 seconds ago    Up 7 seconds    0.0.0.0:3000->30
00/tcp, :::3000->3000/tcp   node-container
09986609848b   my-java-app:v1   "/__cacert_entrypoin…"   10 minutes ago   Up 10 minutes   0.0.0.0:8082->80
80/tcp, :::8082->8080/tcp   java-container
[root@ip-172-31-30-131 NodeJs_based_source]# docker tag my-node-app:v1 \
rajkumari/my-node-app:v1
[root@ip-172-31-30-131 NodeJs_based_source]# docker tag my-node-app:v1 rajkumari2010/my-node-app:v1
[root@ip-172-31-30-131 NodeJs_based_source]# docker push rajkumari2010/my-node-app:v1
The push refers to repository [docker.io/rajkumari2010/my-node-app]
e7bbf23936e8: Pushed
b69872f033fc: Pushed
b0e129a65914: Pushed
41bc30b2a377: Pushed
afa543f85b46: Mounted from library/node
e10358715ead: Mounted from library/node
4983b93ee796: Mounted from library/node
29df493baa13: Mounted from library/node
v1: digest: sha256:b0dedad46e6950e7e5a721ea6b98af5289ec40c70da0b3b2393f2e357c8810e9 size: 1998
[root@ip-172-31-30-131 NodeJs_based_source]#
```





Part 5 — WordPress + MySQL Using Docker Compose

Now create a separate directory:

mkdir wordpress-compose

cd wordpress-compose

vi compose.yaml

root@ip-172-31-30-131:~/wordpress-compose

```
services:

  db:
    image: mysql:8.0
    container_name: wordpress-db

    restart: unless-stopped

    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: wordpress_password
      MYSQL_ROOT_PASSWORD: root_password

    volumes:
      - db_data:/var/lib/mysql


  wordpress:
    image: wordpress:latest
    container_name: wordpress

    restart: unless-stopped

    ports:
      - "8085:80"

    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress_password
      WORDPRESS_DB_NAME: wordpress

    depends_on:
      - db

    volumes:
      - wordpress_data:/var/www/html


volumes:

  db_data:

  wordpress_data:
~
~
```

Step 24 — Start WordPress

Run:

docker compose up -d

You should see:

[+] Running 2/2

✓ Container wordpress-db

✓ Container wordpress

Step 25 — Check Containers

docker compose ps

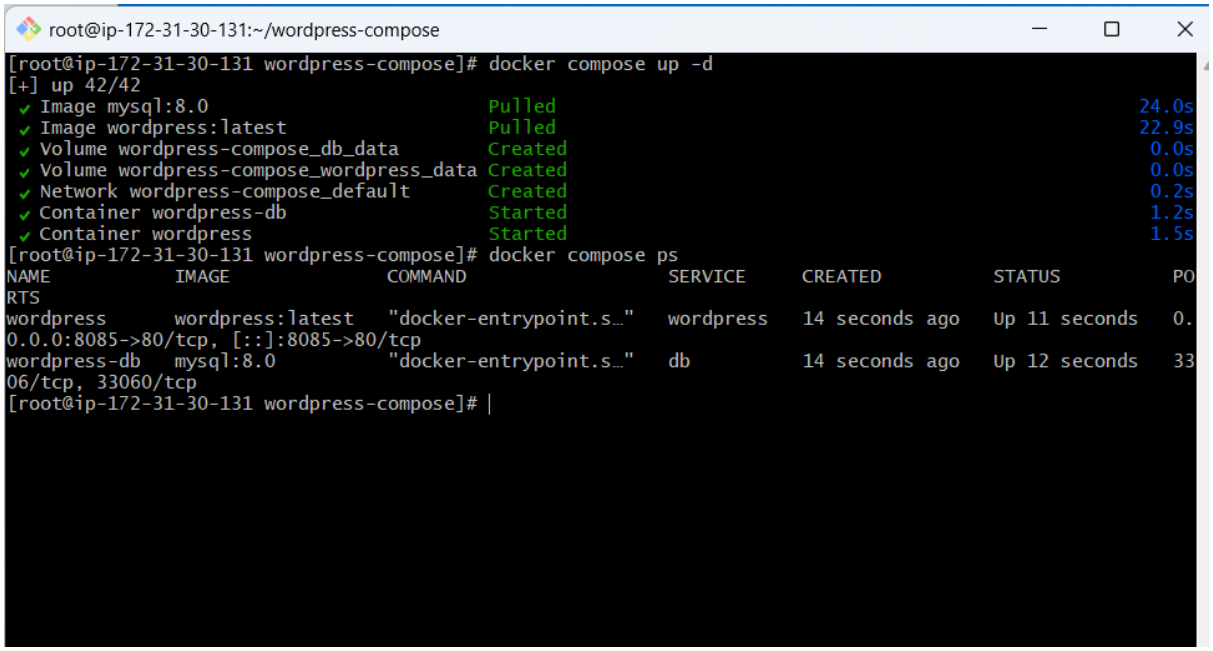
You should have:

wordpress-db

wordpress

Or:

docker ps



Step 26 — Check Logs

WordPress:

docker compose logs wordpress

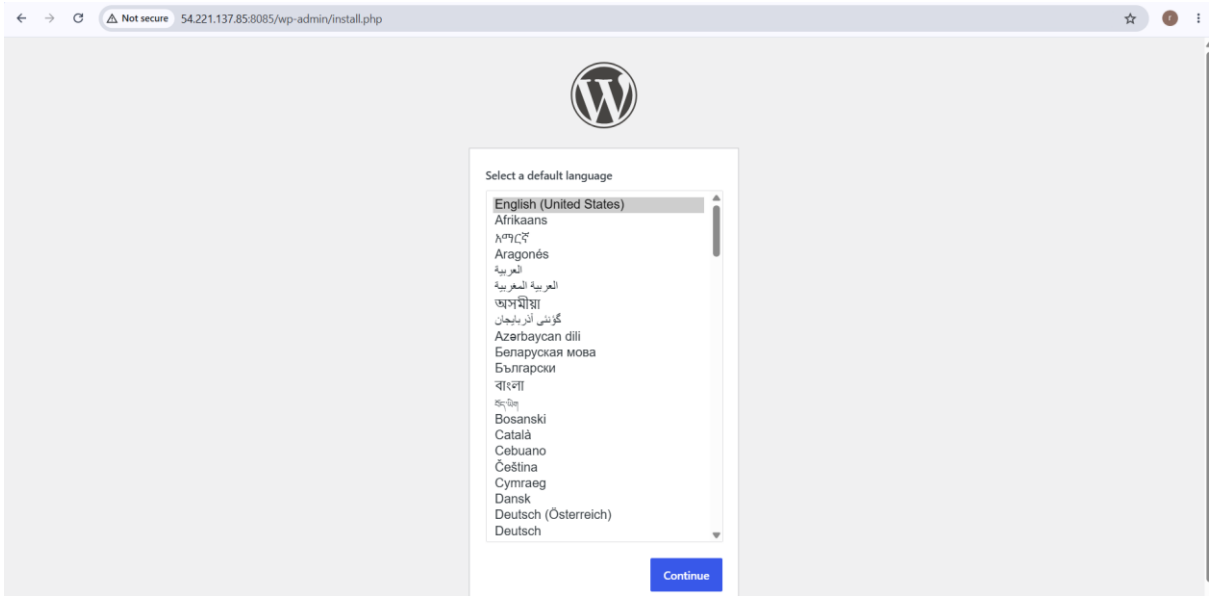
MySQL:

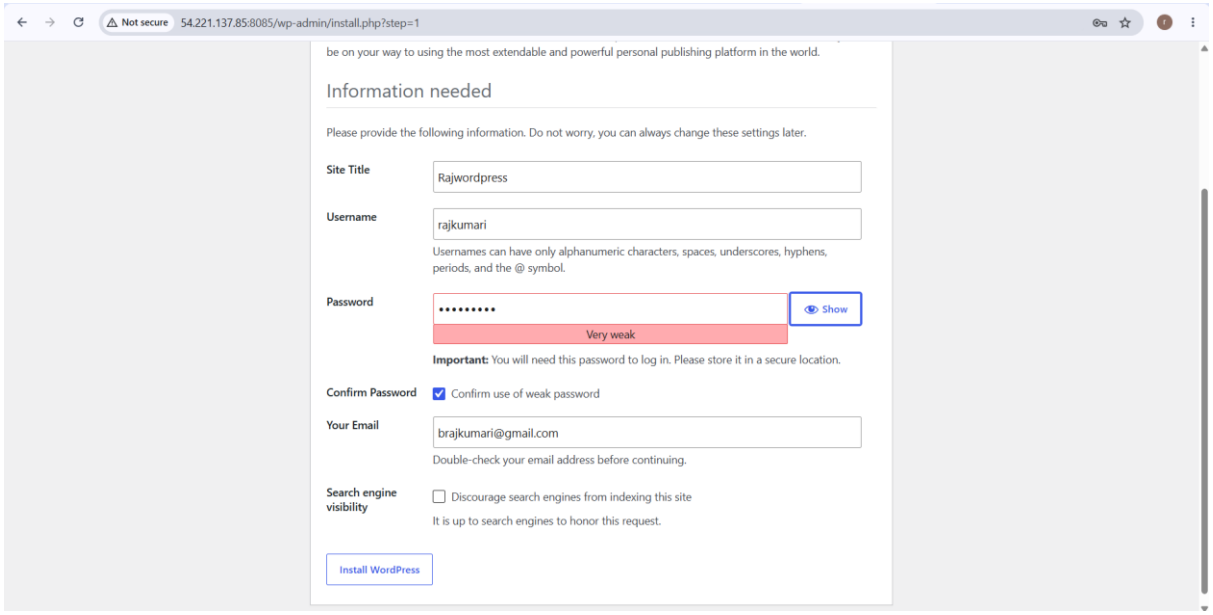
docker compose logs db

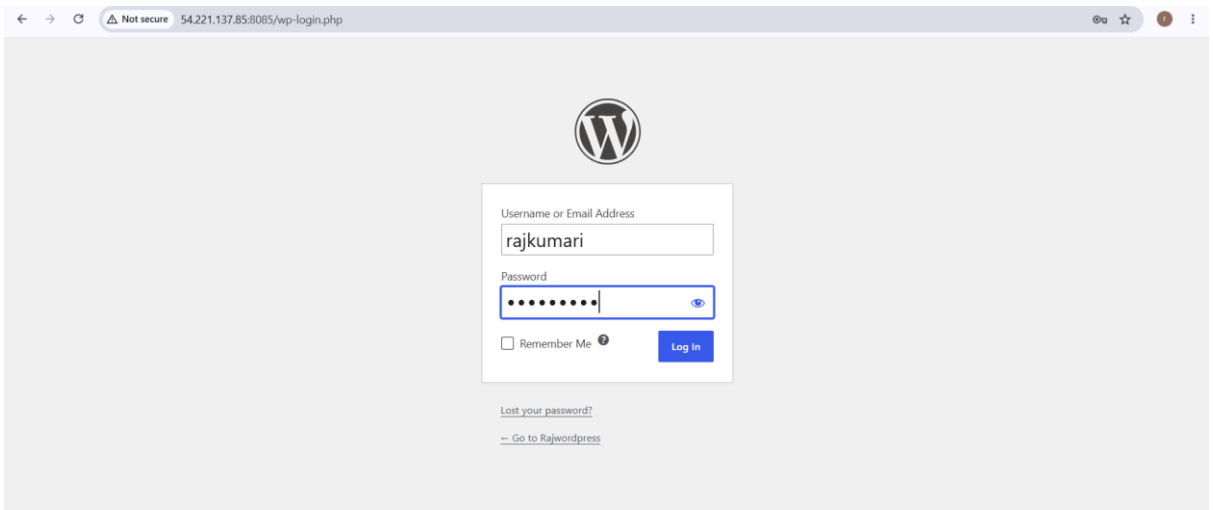
If MySQL is still initializing, wait a little and check again.

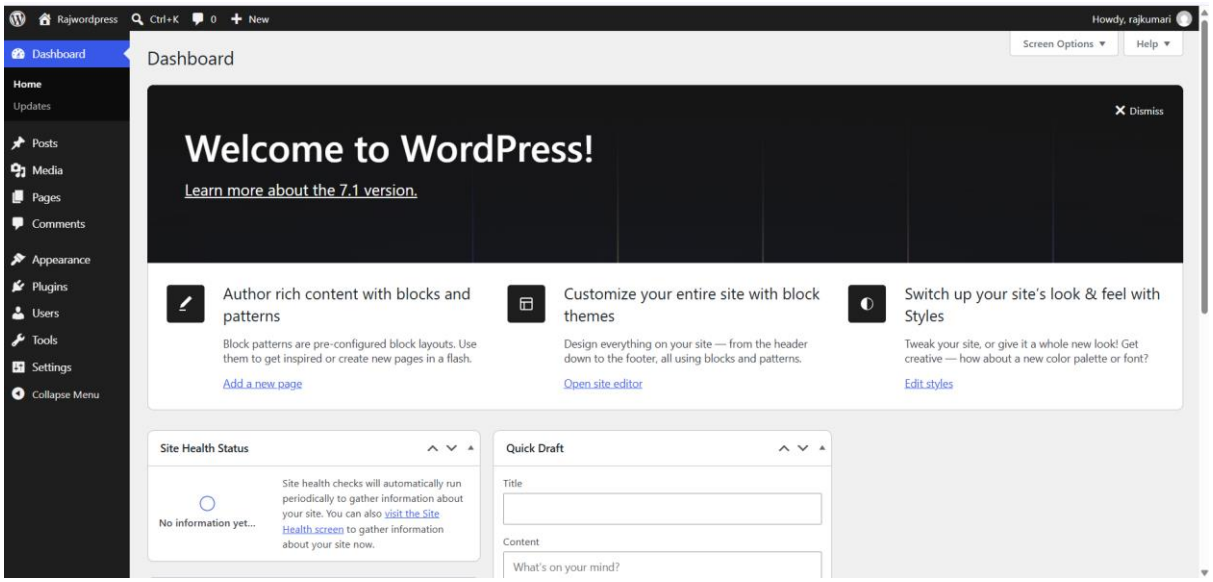
Step 27 — Open WordPress

Open: http://<SERVER-IP>:8085


Not secure
54.221.137.85:8085/wp-admin/install.php
Select a default language
English (United States)
Afrikaans
አማርኛ
Aragonés
العربية
العربية المغربية
অসমীয়া
گؤنئی آذربایجان
Azərbaycan dili
Беларуская мова
Български
বাংলা
Bosanski
Català
Cebuano
Čeština
Cymraeg
Dansk
Deutsch (Österreich)
Deutsch
Continue


Not secure
54.221.137.85:8085/wp-admin/install.php?step=1
be on your way to using the most extendable and powerful personal publishing platform in the world.
Information needed
Please provide the following information. Do not worry, you can always change these settings later.
Site Title
Rajwordpress
Username
rajkumari
Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.
Password
Show
Very weak
Important: You will need this password to log in. Please store it in a secure location.
Confirm Password
Confirm use of weak password
Your Email
brajkumari@gmail.com
Double-check your email address before continuing.
Search engine visibility
Discourage search engines from indexing this site
It is up to search engines to honor this request.
Install WordPress


Not secure
54.221.137.85:8085/wp-login.php
Username or Email Address
rajkumari
Password
Remember Me
Log In
Lost your password?
← Go to Rajwordpress

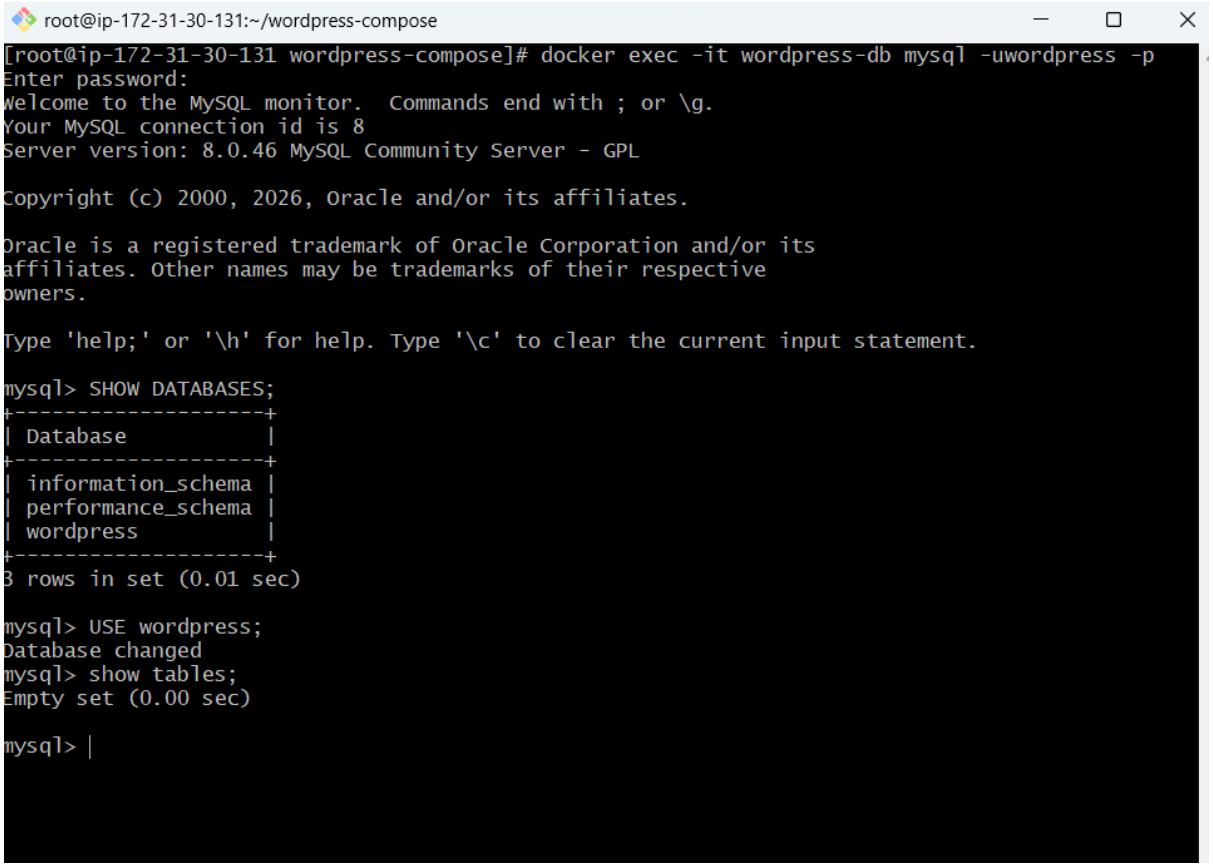


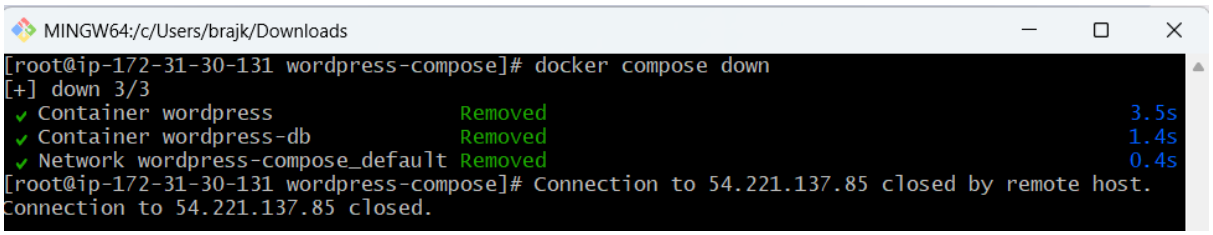
Step 28 — Verify Docker Compose Networking

Notice this:

WORDPRESS_DB_HOST: db:3306

```
root@ip-172-31-30-131:~/wordpress-compose
[root@ip-172-31-30-131 wordpress-compose]# docker compose up -d
[+] up 2/2
 ✔ Container wordpress     Running                                                0.0s
 ✔ Container wordpress-db  Running                                                0.0s
[root@ip-172-31-30-131 wordpress-compose]# docker compose ps
NAME           IMAGE              COMMAND                  SERVICE     CREATED          STATUS          PO
RTS
wordpress      wordpress:latest   "docker-entrypoint.s…"   wordpress   14 minutes ago   Up 14 minutes   0.
0.0.0:8085->80/tcp, [::]:8085->80/tcp
wordpress-db   mysql:8.0          "docker-entrypoint.s…"   db          14 minutes ago   Up 14 minutes   33
06/tcp, 33060/tcp
[root@ip-172-31-30-131 wordpress-compose]# docker compose logs db
wordpress-db  | 2026-08-27 18:02:17+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1
.el9 started.
wordpress-db  | 2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
wordpress-db  | 2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.46-1
.el9 started.
wordpress-db  | 2026-08-27 18:02:18+00:00 [Note] [Entrypoint]: Initializing database files
wordpress-db  | 2026-08-27T18:02:18.764749Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache
' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
wordpress-db  | 2026-08-27T18:02:18.765694Z 0 [System] [MY-013169] [Server] /usr/sbin/mysqld (mysqld 8.0.4
6) initializing of server in progress as process 79
wordpress-db  | 2026-08-27T18:02:18.788000Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has star
ted.
wordpress-db  | 2026-08-27T18:02:19.489828Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ende
d.
wordpress-db  | 2026-08-27T18:02:21.108595Z 6 [Warning] [MY-010453] [Server] root@localhost is created wit
h an empty password ! Please consider switching off the --initialize-insecure option.
wordpress-db  | 2026-08-27 18:02:24+00:00 [Note] [Entrypoint]: Database files initialized
wordpress-db  | 2026-08-27 18:02:24+00:00 [Note] [Entrypoint]: Starting temporary server
wordpress-db  | 2026-08-27T18:02:25.342487Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache
' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
wordpress-db  | 2026-08-27T18:02:25.345191Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.4
6) starting as process 121
wordpress-db  | 2026-08-27T18:02:25.376995Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has star
ted.
wordpress-db  | 2026-08-27T18:02:25.906470Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ende
d.
wordpress-db  | 2026-08-27T18:02:26.380785Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self
 signed.
wordpress-db  | 2026-08-27T18:02:26.380835Z 0 [System] [MY-013602] [Server] Channel mysql_main configured
to support TLS. Encrypted connections are now supported for this channel.
wordpress-db  | 2026-08-27T18:02:26.389384Z 0 [Warning] [MY-011810] [Server] Insecure configuration for --
pid-file: Location '/var/run/mysqld' in the path is accessible to all OS users. Consider choosing a differ
ent directory.
wordpress-db  | 2026-08-27T18:02:26.429013Z 0 [System] [MY-011323] [Server] X Plugin ready for connections
. Socket: /var/run/mysqld/mysqlx.sock
wordpress-db  | 2026-08-27T18:02:26.429379Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for co
nnections. Version: '8.0.46'  socket: '/var/run/mysqld/mysqld.sock'  port: 0  MySQL Community Server - GPL
.
wordpress-db  | 2026-08-27 18:02:26+00:00 [Note] [Entrypoint]: Temporary server started.
wordpress-db  | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
```





Step 29 — Stop WordPress

When finished:

docker compose down

This removes the containers but keeps named volumes.

Therefore your database data remains in:

db_data

and WordPress data remains in:

wordpress_data

Step 30 — Remove Everything Including Data

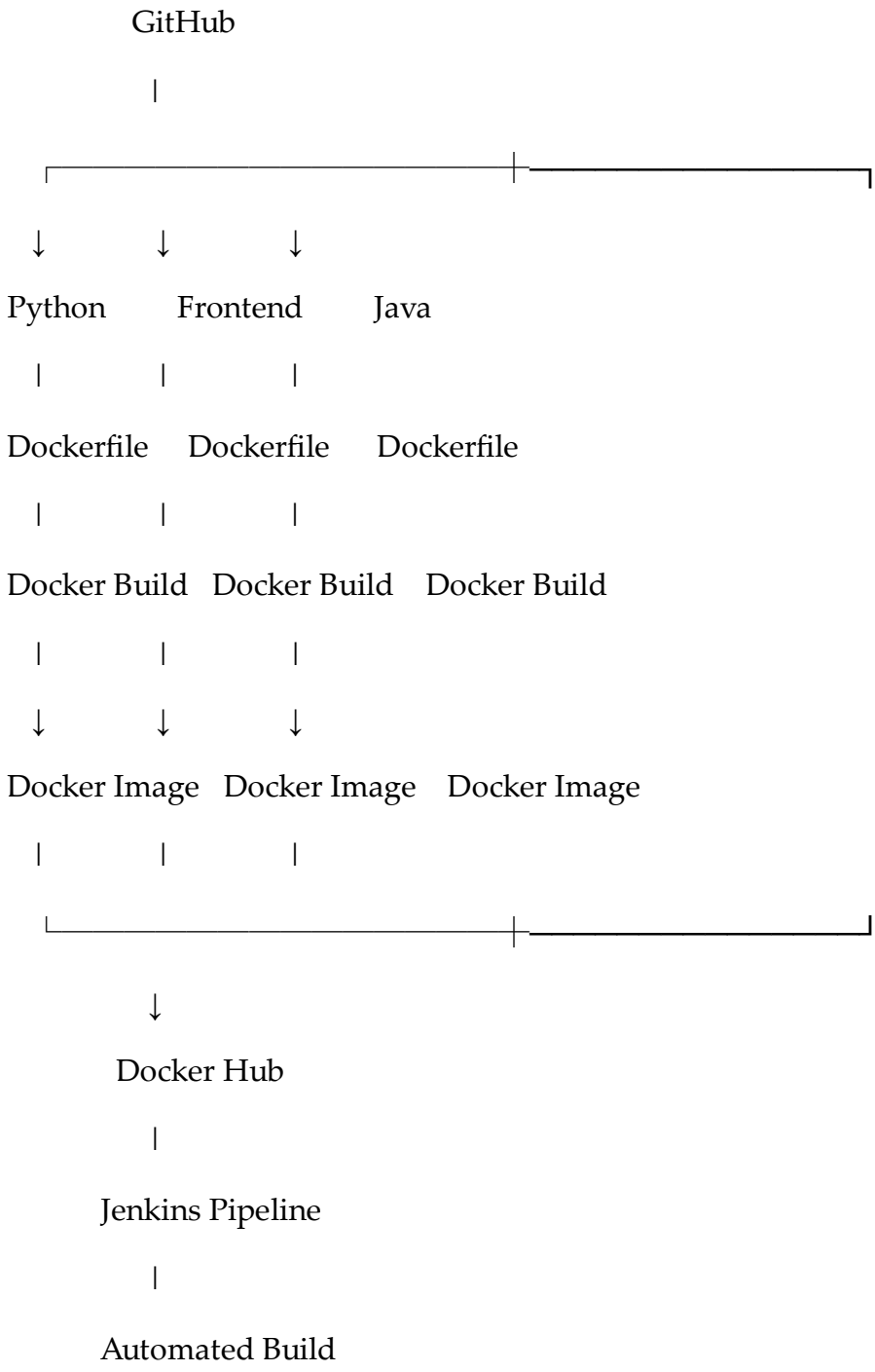
If you want to completely remove the assignment:

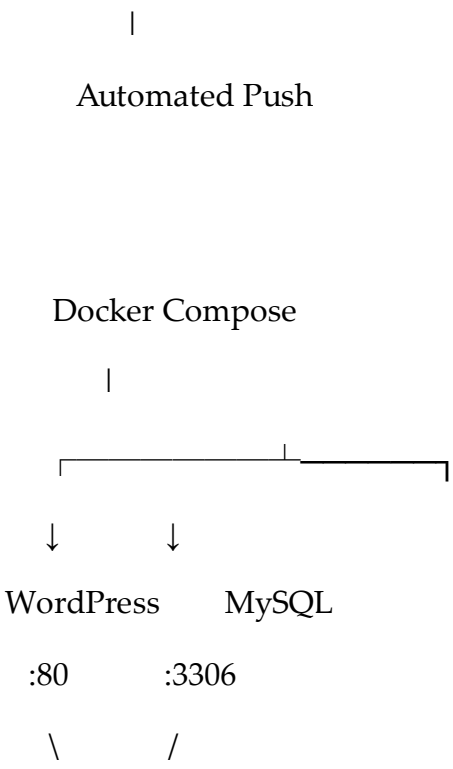
docker compose down -v

Be careful: -v deletes the named volumes and therefore your WordPress/MySQL data.

Final Assignment Structure

Your complete practical can be documented as:





Commands Cheat Sheet

Python/Jenkins

git clone https://github.com/betawins/Python-app.git

docker build -t python-app:latest .

docker run -d -p 5000:5000 python-app:latest

docker tag python-app:latest USERNAME/python-app:latest

docker push USERNAME/python-app:latest

Frontend

docker build -t frontend:v1 .

docker run -d -p 8081:80 frontend:v1

docker tag frontend:v1 USERNAME/frontend:v1

docker push USERNAME/frontend:v1

Java

docker build -t java-app:v1 .

docker run -d -p 8082:8080 java-app:v1

docker tag java-app:v1 USERNAME/java-app:v1

docker push USERNAME/java-app:v1

Node.js

docker build -t node-app:v1 .

docker run -d -p 3000:3000 node-app:v1

docker tag node-app:v1 USERNAME/node-app:v1

docker push USERNAME/node-app:v1

WordPress + MySQL

docker compose config

docker compose up -d

docker compose ps

docker compose logs

docker compose down

Jenkins Pipeline Flow

GitHub

↓

Git Clone

↓

Docker Build

↓

Docker Login

↓

Docker Push

↓

Docker Hub

One important point: for the frontend, Java, and Node.js tasks, the exact directory names and application ports in docker-tasks should be verified from the repository before you create the Dockerfiles. I don't want to give you a Dockerfile that assumes the wrong source directory or port. The repository is the correct starting point, but its current top-level listing isn't exposing the task subdirectories in the web result.

If you are doing this on the **Jenkins EC2 you just created**, the next practical step is **Task 1: create the Jenkins job for Python-app**. Start with the Docker/Jenkins prerequisites, then create the pipeline above.